

Question 1

Shape Analysis on Human Hand Shapes.

```
def get_preshape(image): # need this fun for code11 (GPA)
    image = image - np.mean(image, axis=0) #shape=(2,), Transpose otherwise we
might lose x,y
    image = image / np.linalg.norm(image)
    return image

def code1(mean_shape, shape):
    A = shape.T @ mean_shape
    U, _, V = np.linalg.svd(A)
    R = V.T @ U.T

    if np.linalg.det(R) < 0: # to make sure we don't get negative deter
(reflection)
        V.T[-1,:] *= -1
        R = V.T @ U.T
    return shape @ R

def code2(shape1, shape2):
    shape1 = shape1 - np.mean(shape1, axis=0) # translate
    shape2 = shape2 - np.mean(shape2, axis=0)

    A = shape2.T @ shape1
    U, _, V = np.linalg.svd(A)
    R = V.T @ U.T

    if np.linalg.det(R) < 0: # the same thing as above
        V.T[-1,:] *= -1
        R = V.T @ U.T

    scaling = np.trace(shape1.T @ shape2 @ R) / (np.linalg.norm(shape2)**2)
    shape1_mean = np.mean(shape1, axis=0)
    shape2_mean = np.mean(shape2, axis=0)
    t = shape1_mean - scaling * (shape2_mean @ R) # t = mx - s*(R*my)
    return scaling * (shape2 @ R) + t # s*R*Y + t
```

```

def code11(image):
    _, features, num_shapes = image.shape
    cont_shape = list()

    for shape in range(num_shapes):
        X = image[:, :, shape].T
        X = get_preshape(X)
        cont_shape.append(X)

    ini_shape = cont_shape[0]
    for _ in range(100):
        aligned = list()

        for X_new in cont_shape:
            Y = code1(ini_shape, X_new)
            aligned.append(Y)

        new_mean = np.mean(aligned, axis=0)
        new_mean = get_preshape(new_mean)

        if np.linalg.norm(new_mean - ini_shape) < 1e-6:
            break

        ini_shape = new_mean

    return aligned

def code22(image):
    _, features, num_shapes = image.shape
    cont_shape = [ image[:, :, i].T for i in range(num_shapes) ]
    ini_shape = cont_shape[0]

    for _ in range(100):
        aligned = list()
        for X_new in cont_shape:
            Y = code2(ini_shape, X_new)
            aligned.append(Y)

        new_mean = np.mean(aligned, axis=0)

        if np.linalg.norm(new_mean - ini_shape) < 1e-6:
            break

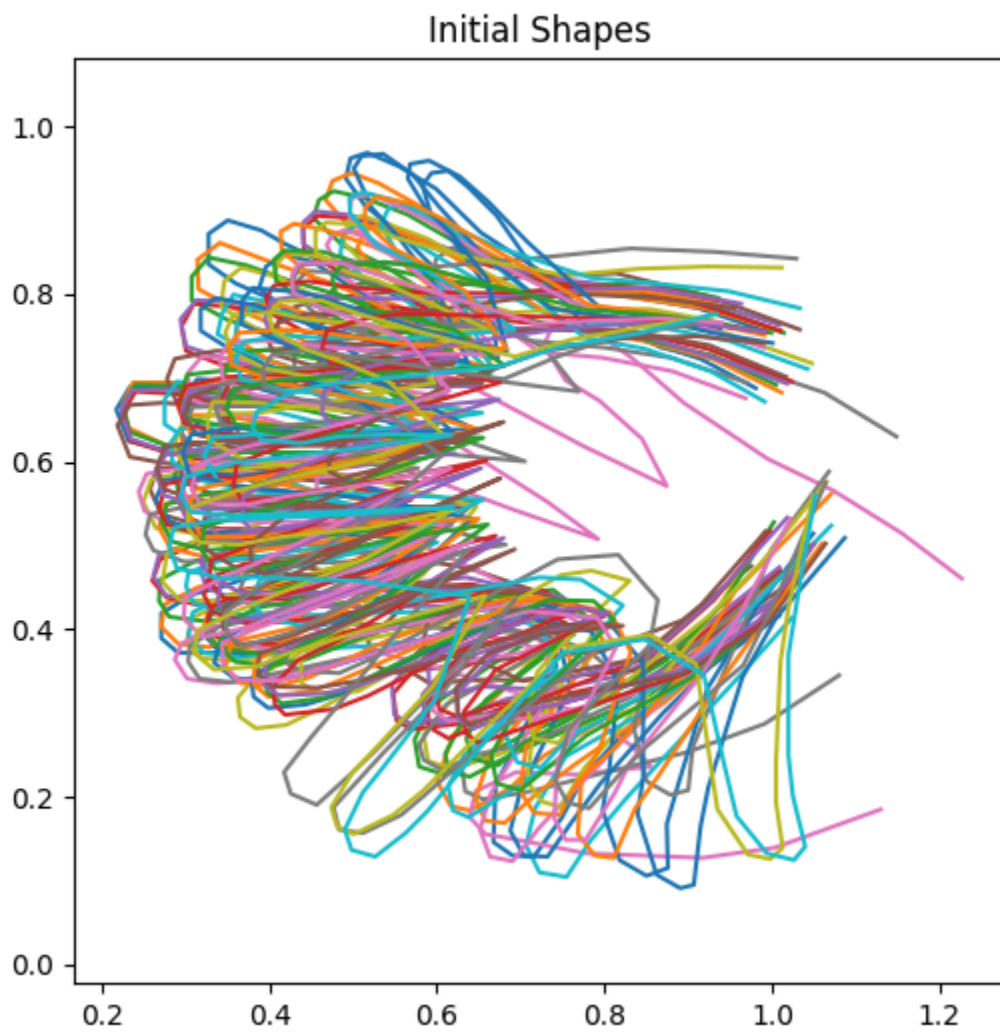
        ini_shape = new_mean

    return aligned

```

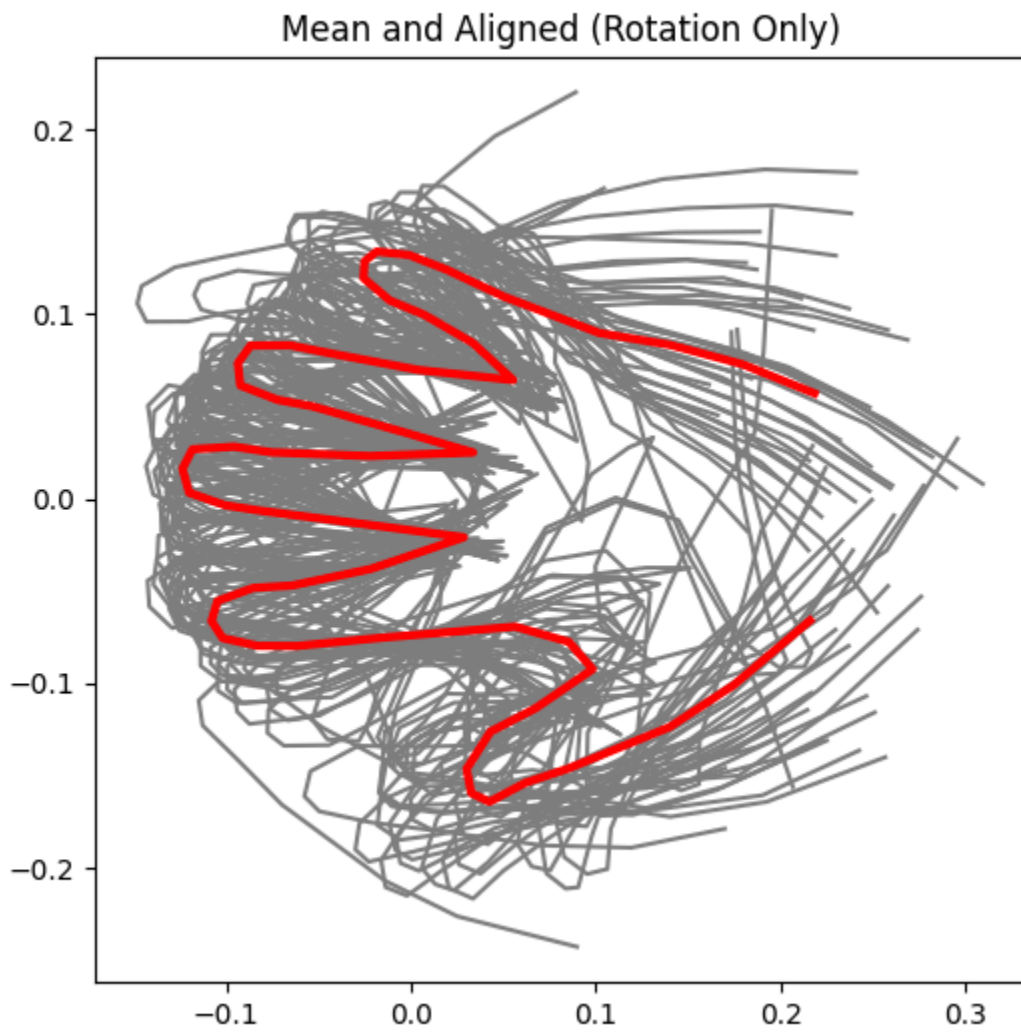
PART A

Plot of Initial pointsets

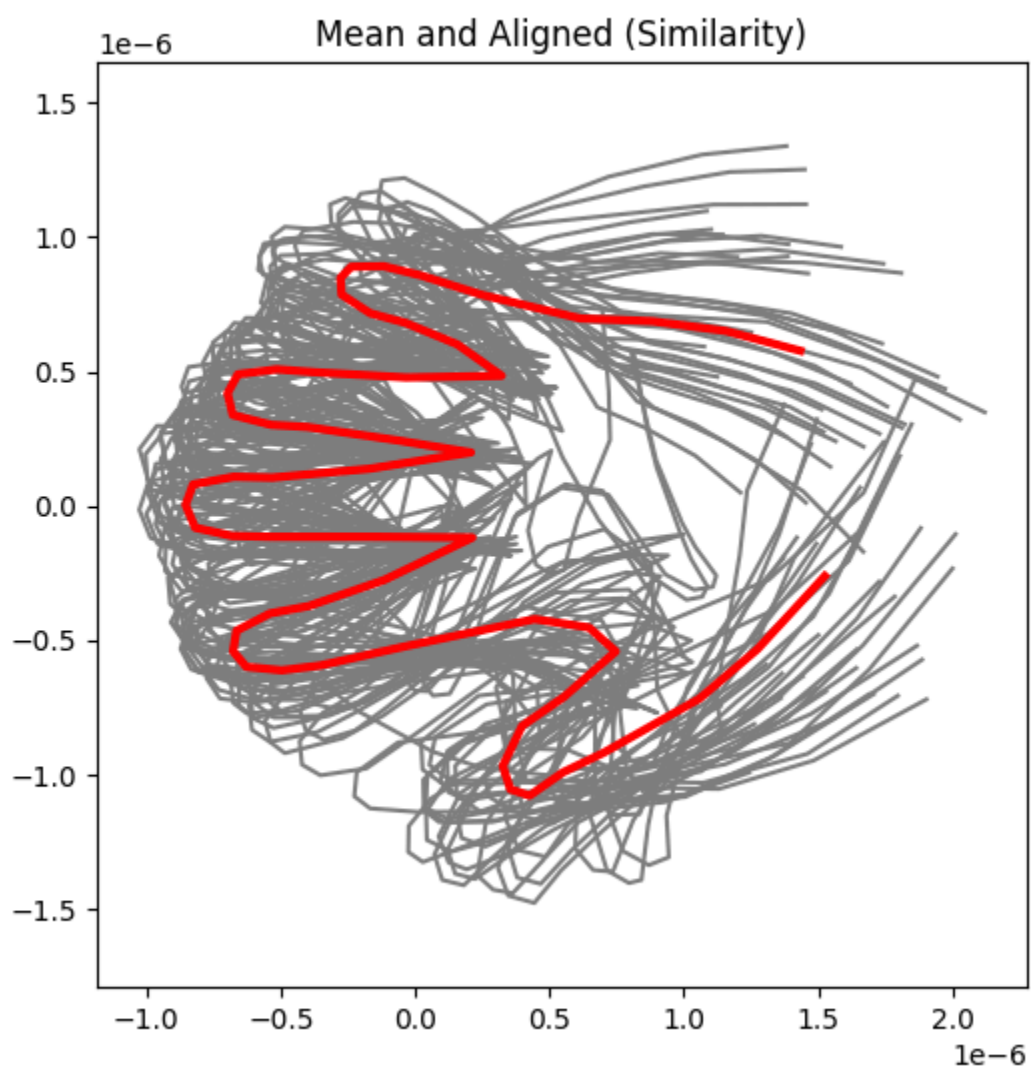


PART B

Code11

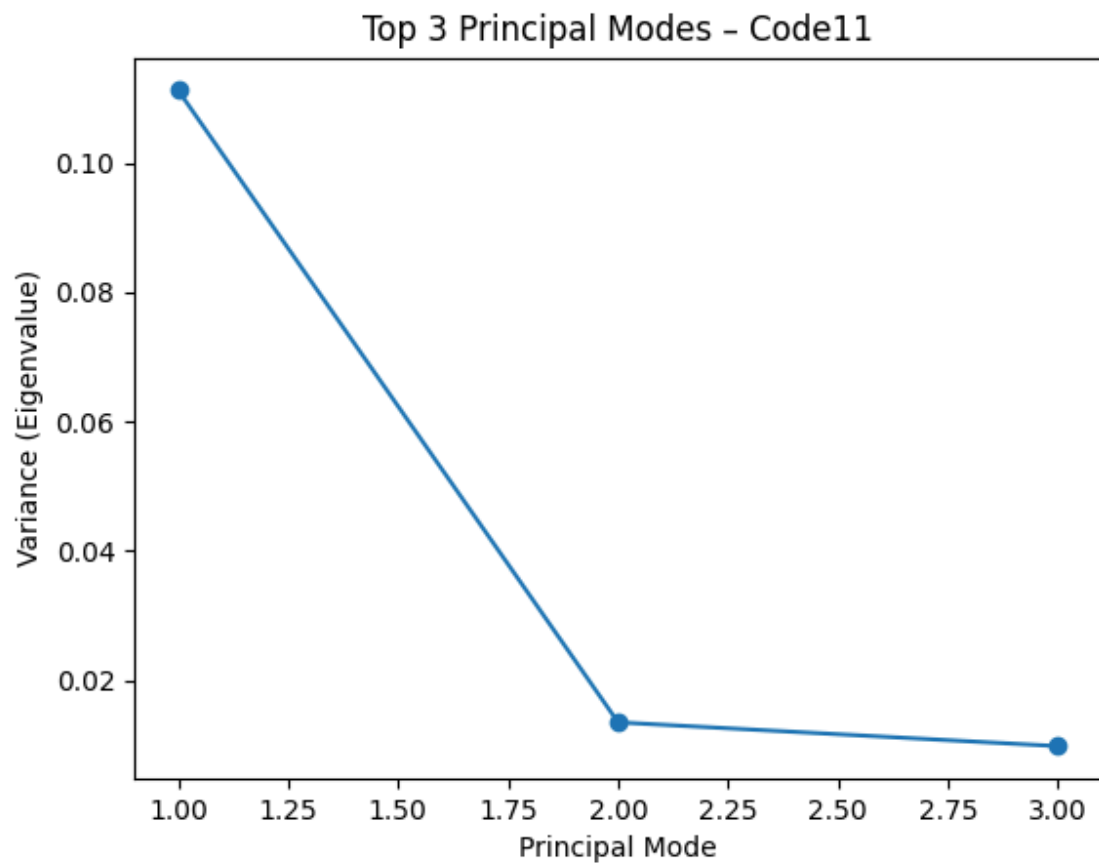


Code22

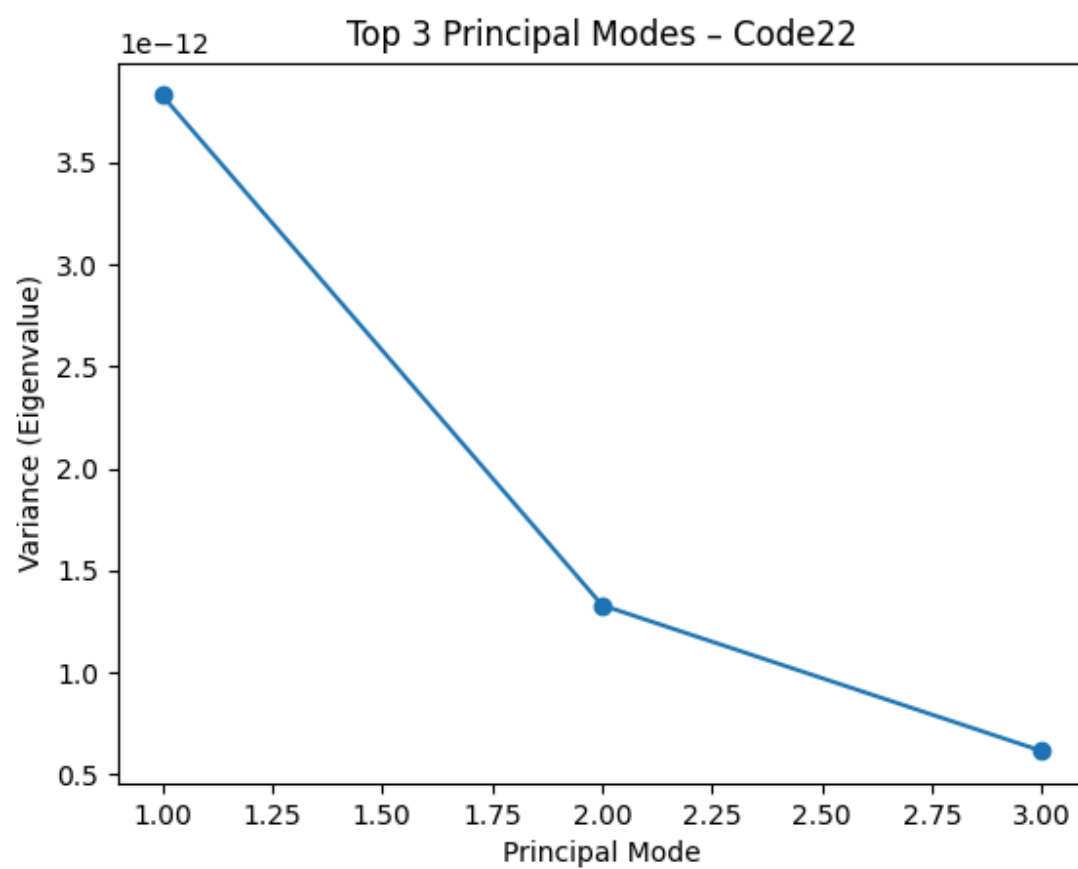


PART C

Code11

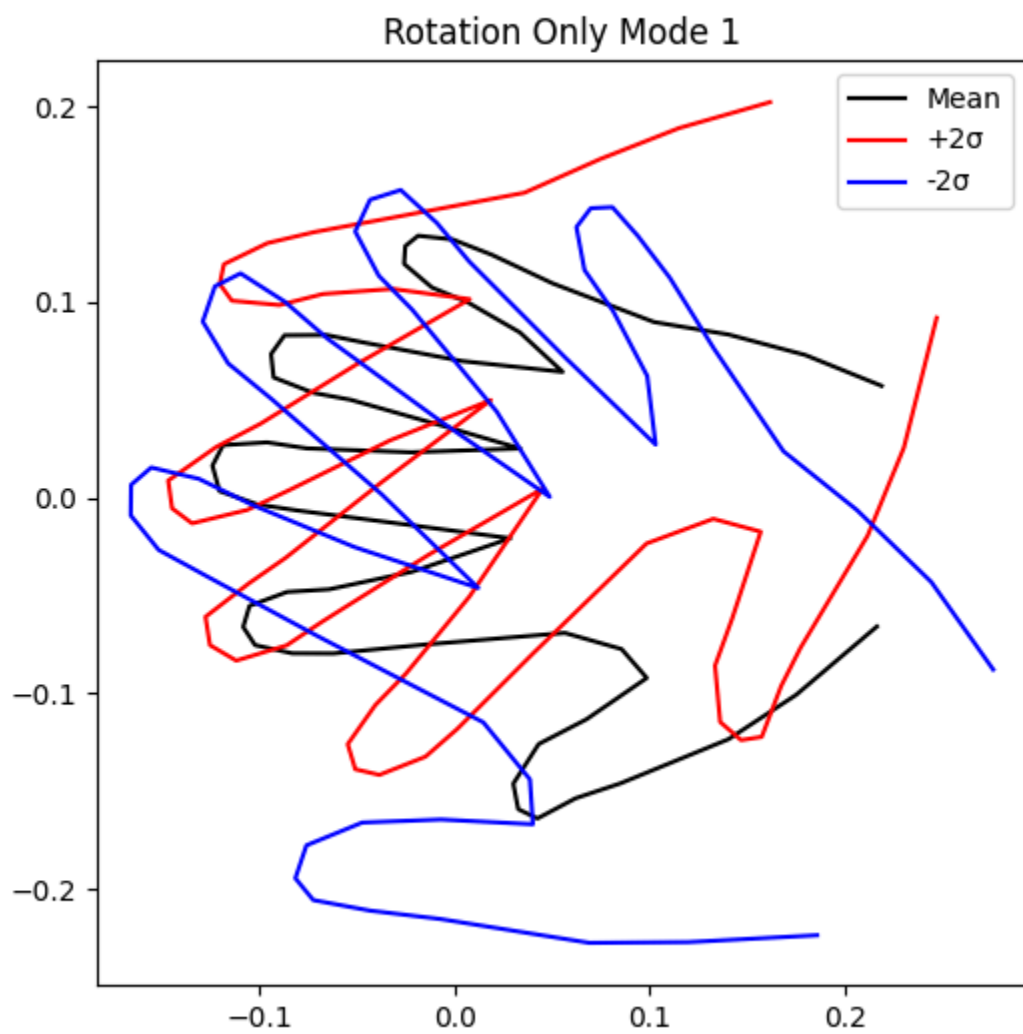


Code22

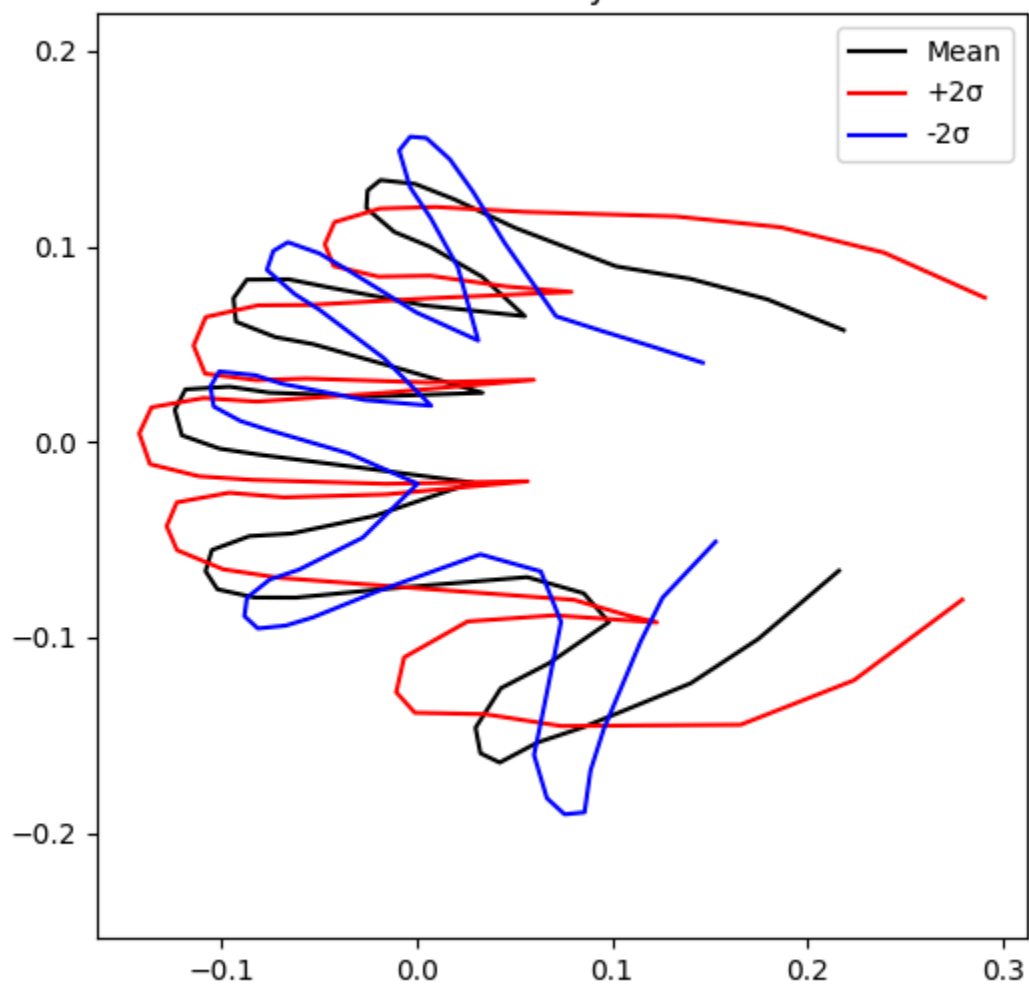


PART D

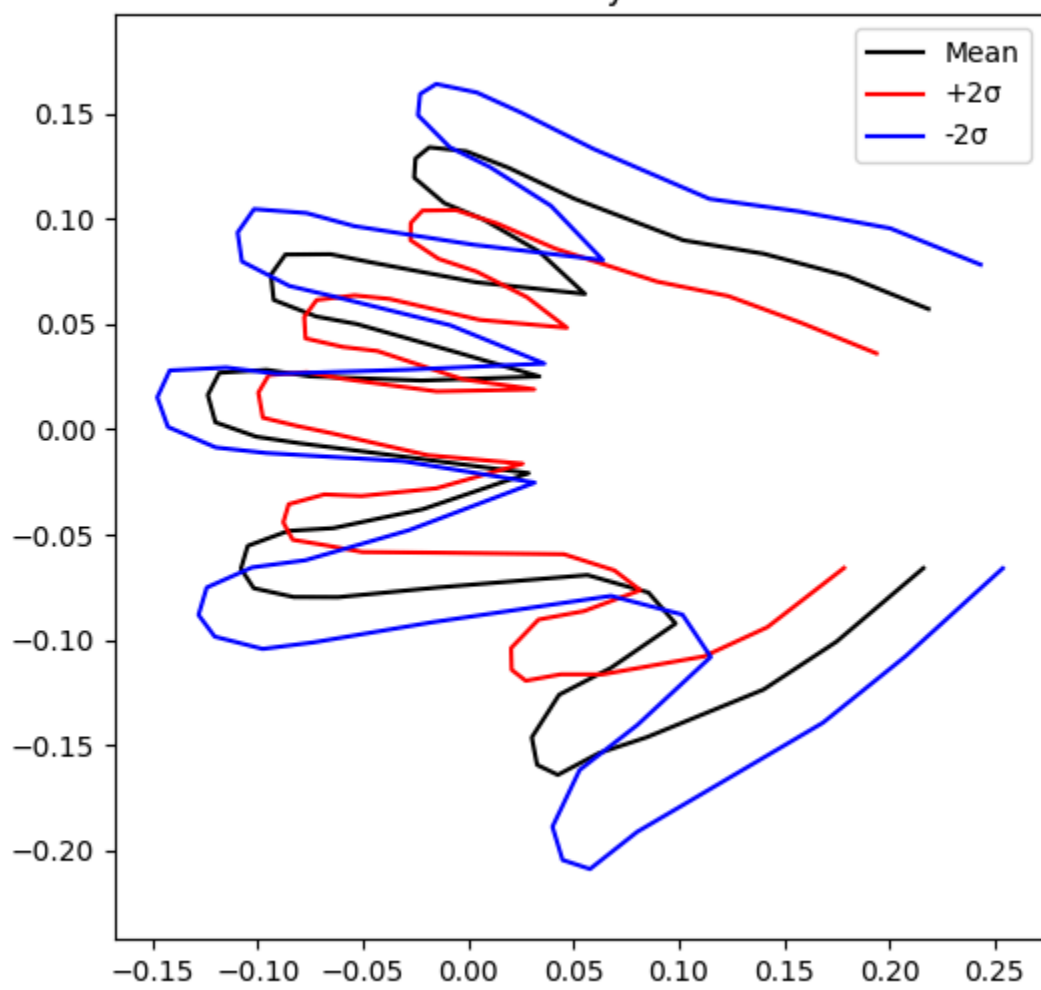
Code11



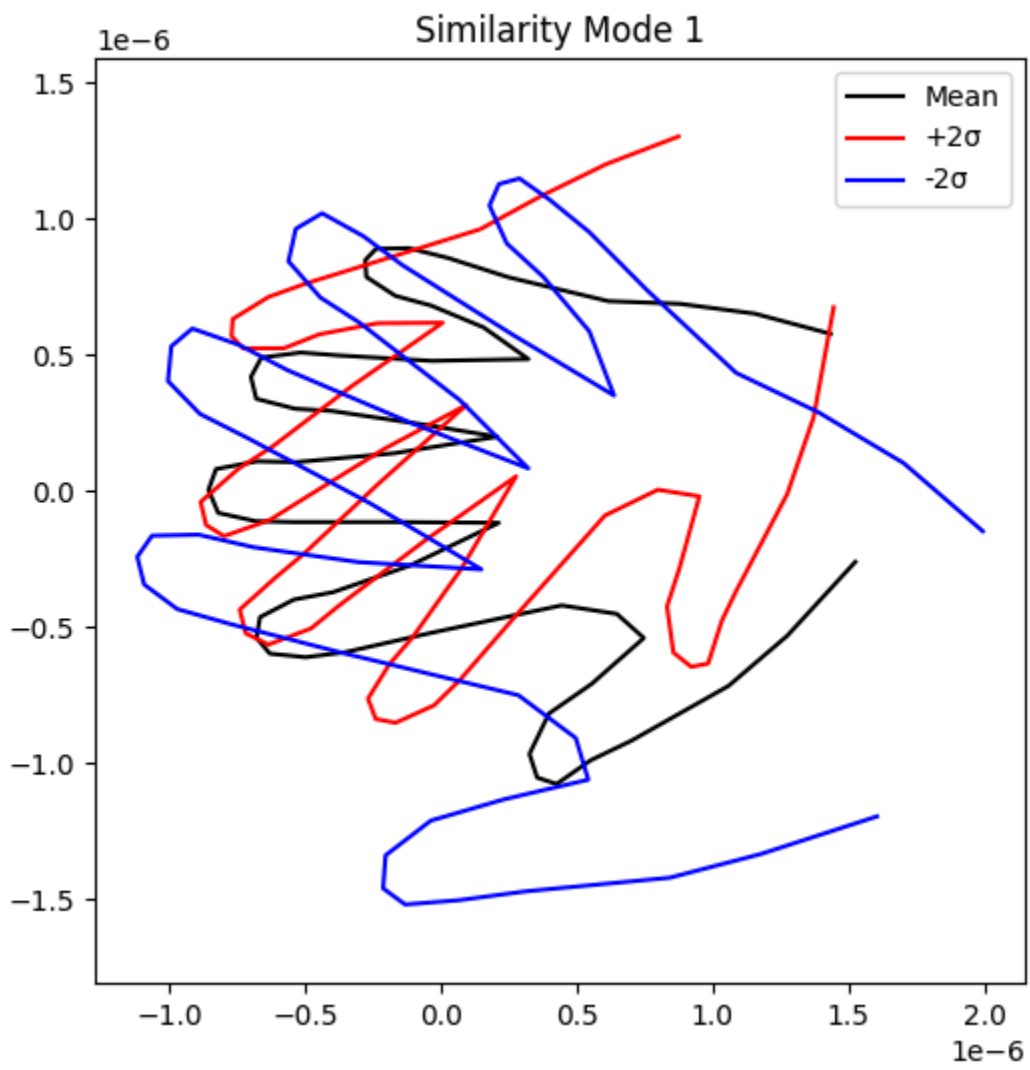
Rotation Only Mode 2

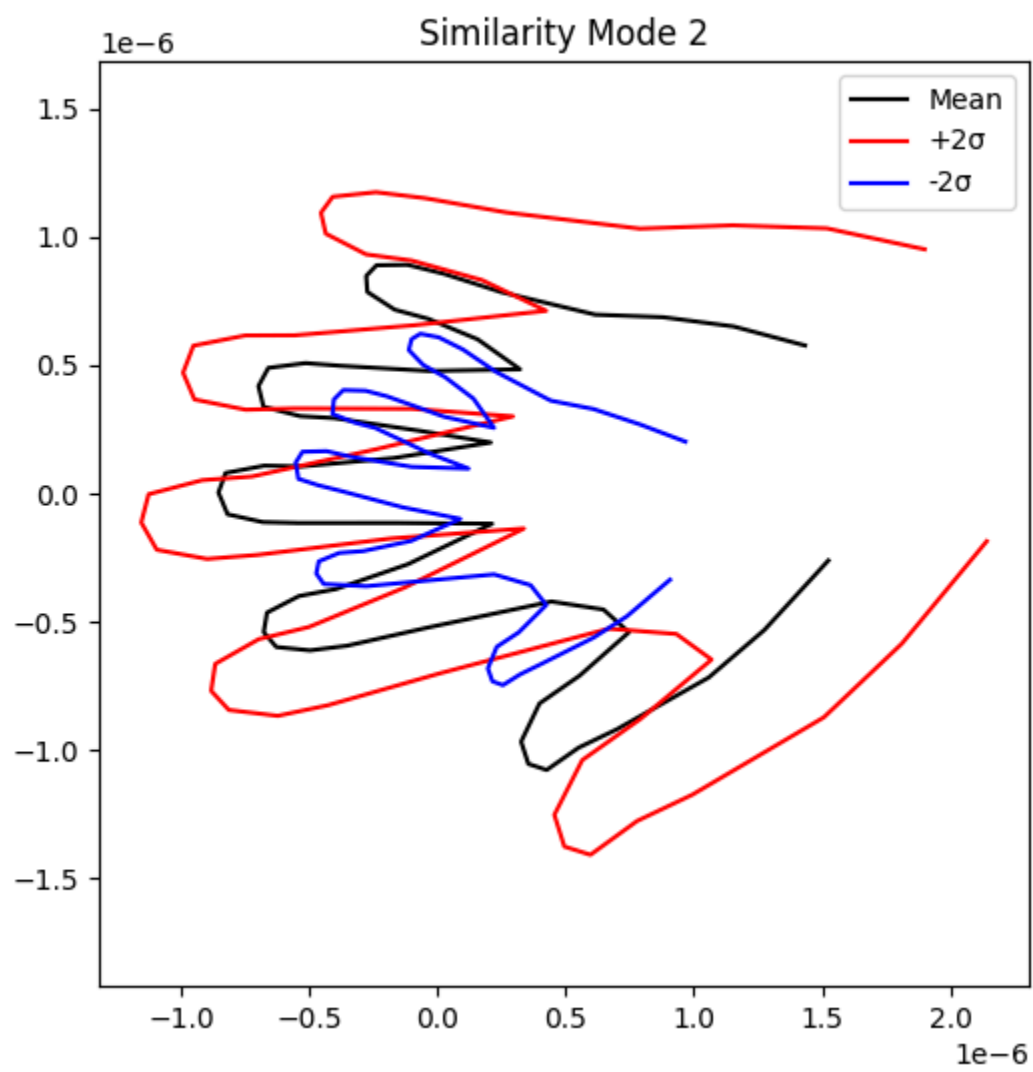


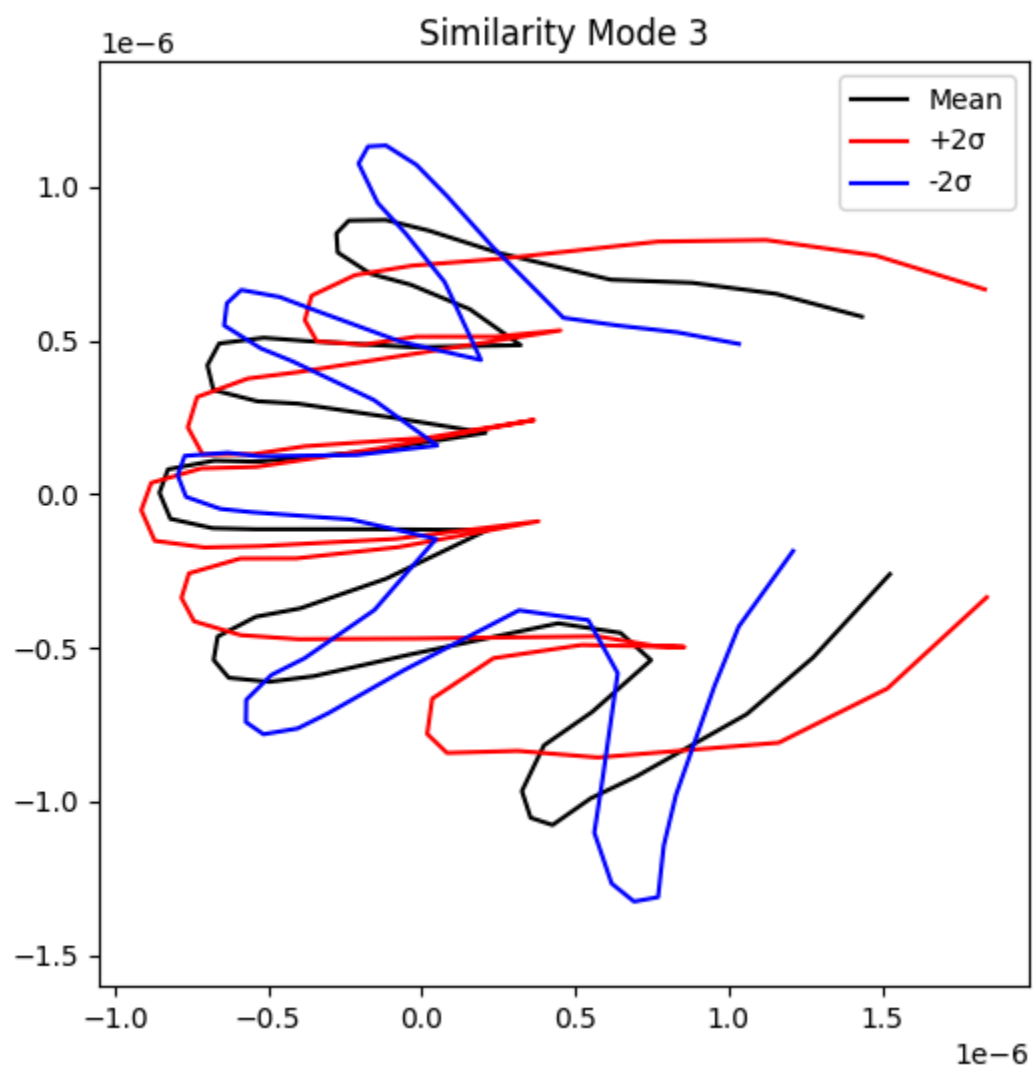
Rotation Only Mode 3



Code22







Question 2

Shape Analysis on Human Cardiac Shapes.

PART A

```
def extract_conts(img):
    contours = measure.find_contours(img, level=125)

    contours = sorted(contours, key=lambda x: len(x), reverse=True) # sort by
the size
    if len(contours) < 2:
        raise ValueError("not enough contours")
    outer = contours[0][:, ::-1] # (y,x) to (x,y)
    inner = contours[1][:, ::-1]
    print(outer.shape)
    print(inner.shape)
    return outer, inner

def resample_curve(points, num_points=50): # tried from 10 to 100, 50 is
sufficient

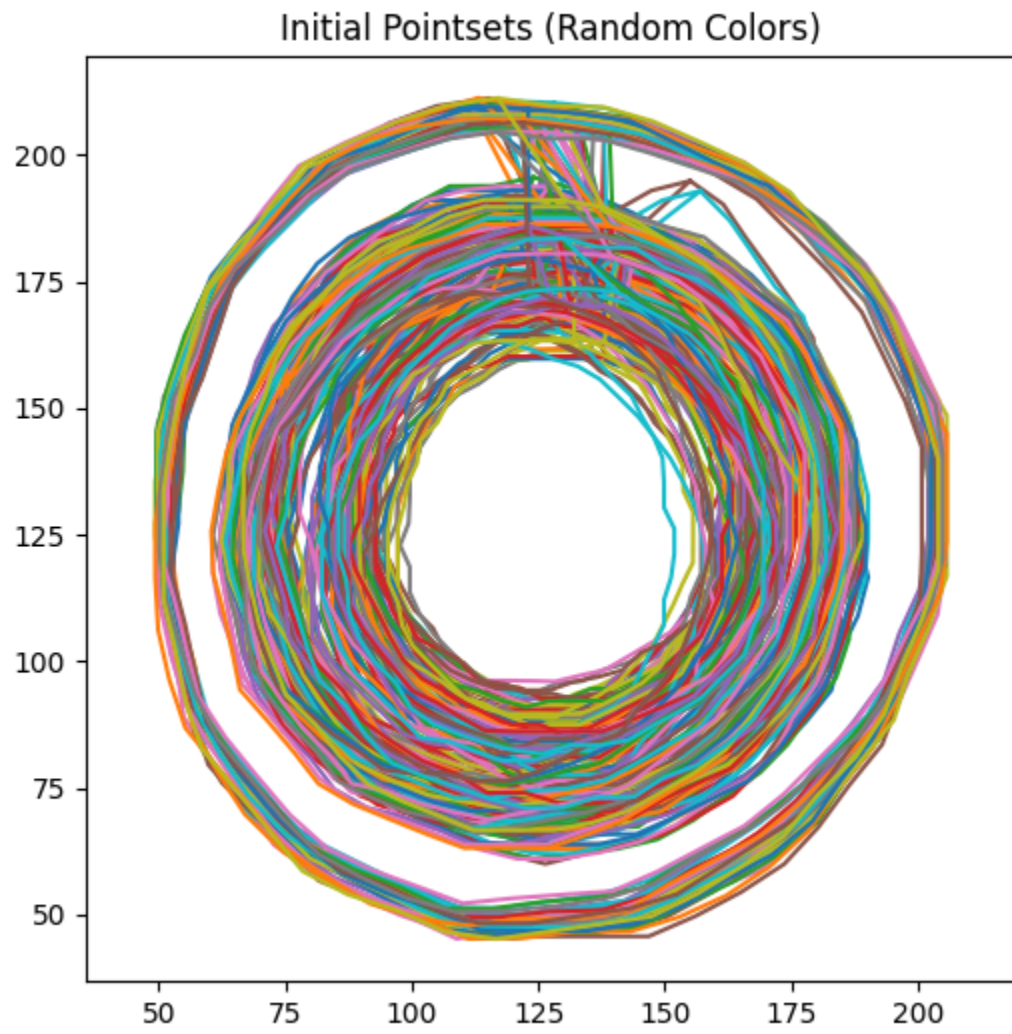
    #d = np.sqrt(((np.diff(points, axis=0))**2).sum(axis=1)) # distance
    #d = np.insert(np.cumsum(d), 0, 0)
    d = np.cumsum(np.r_[0, np.linalg.norm(np.diff(points, axis=0), axis=1)])
    print(len(d))
    new_d = np.linspace(0, d[-1], num_points) # the numbers we want, might
tune later

    new_points = np.zeros((num_points, 2))

    for i in range(2):
        new_points[:, i] = np.interp(new_d, d, points[:, i]) # linear
interpolation

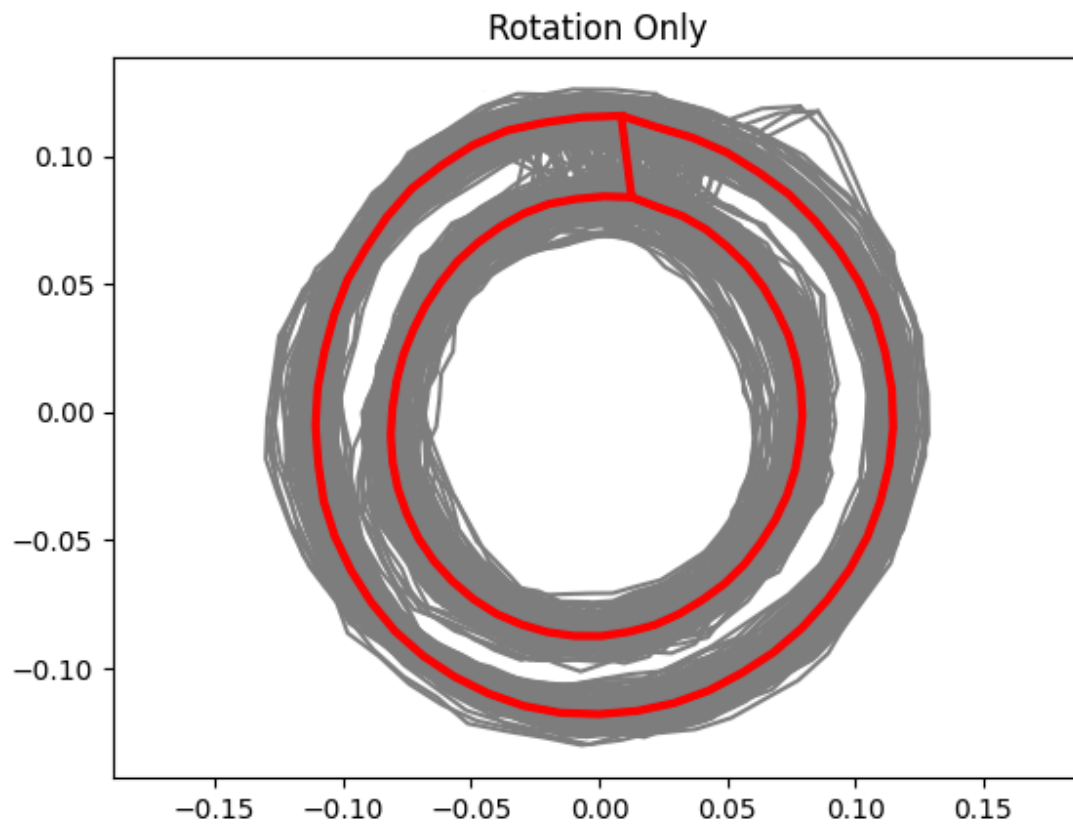
    return new_points
```


PART B

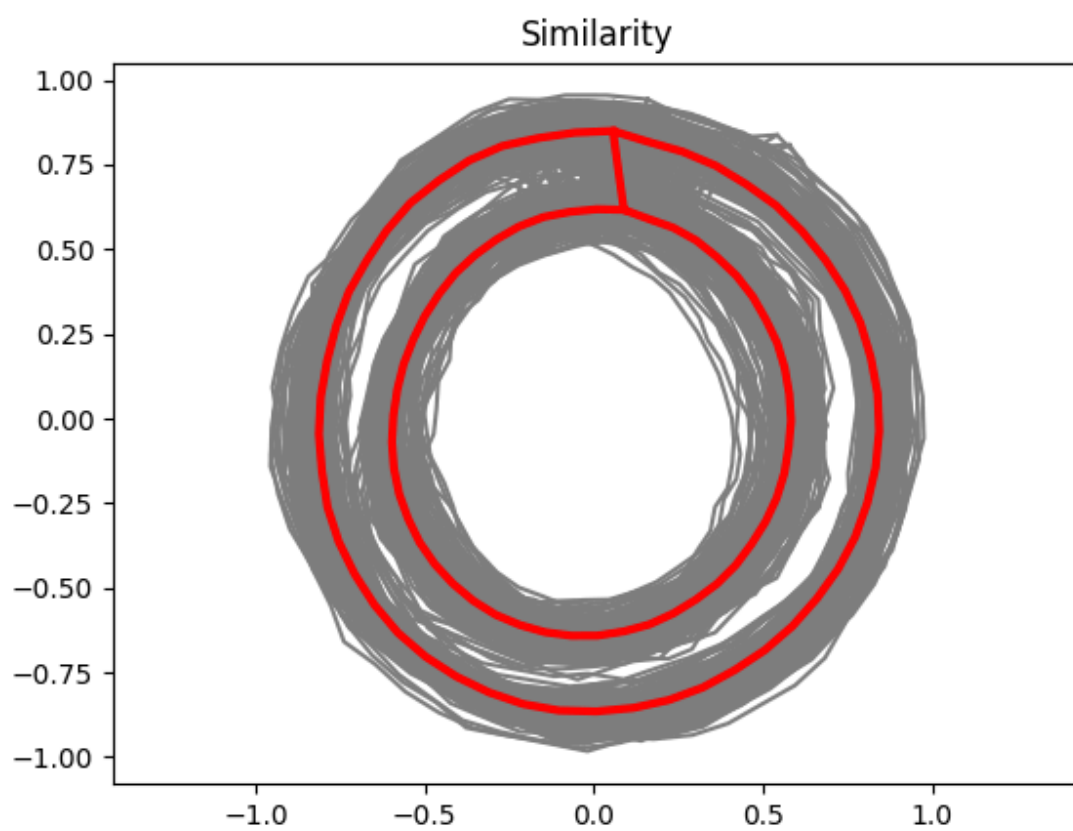


PART C

Code11

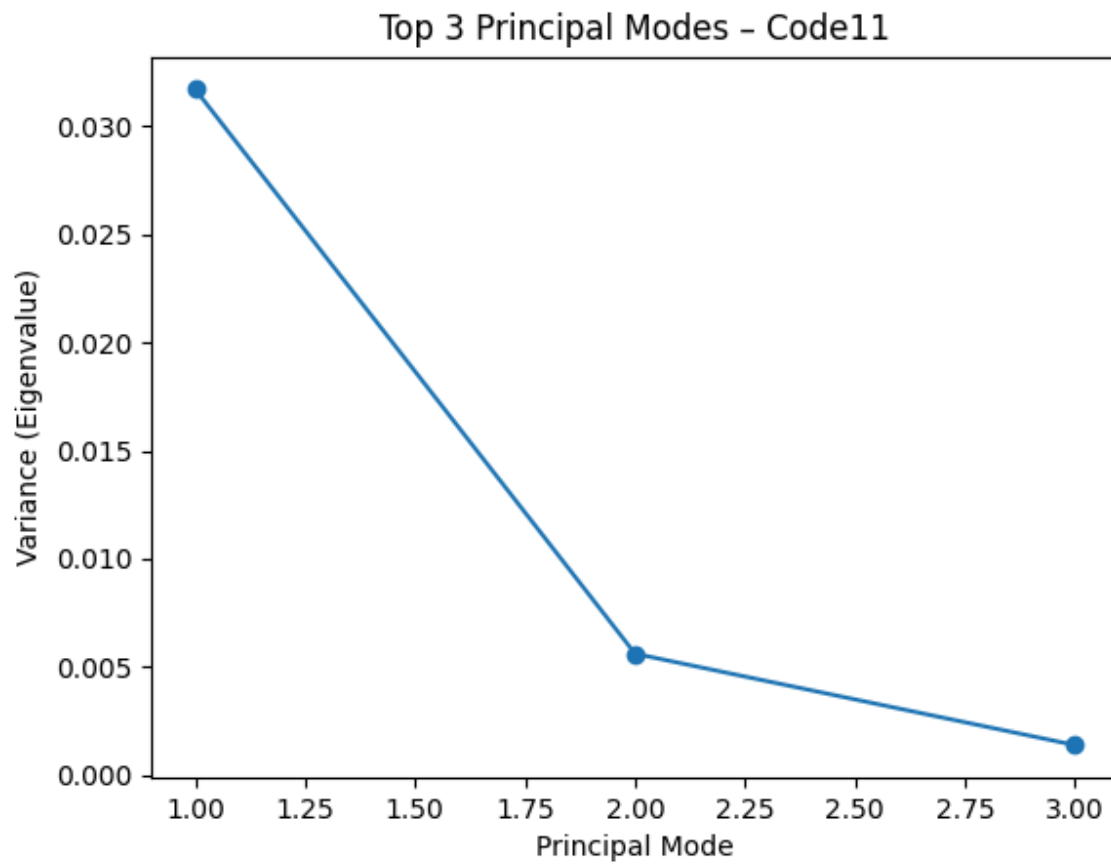


Code22

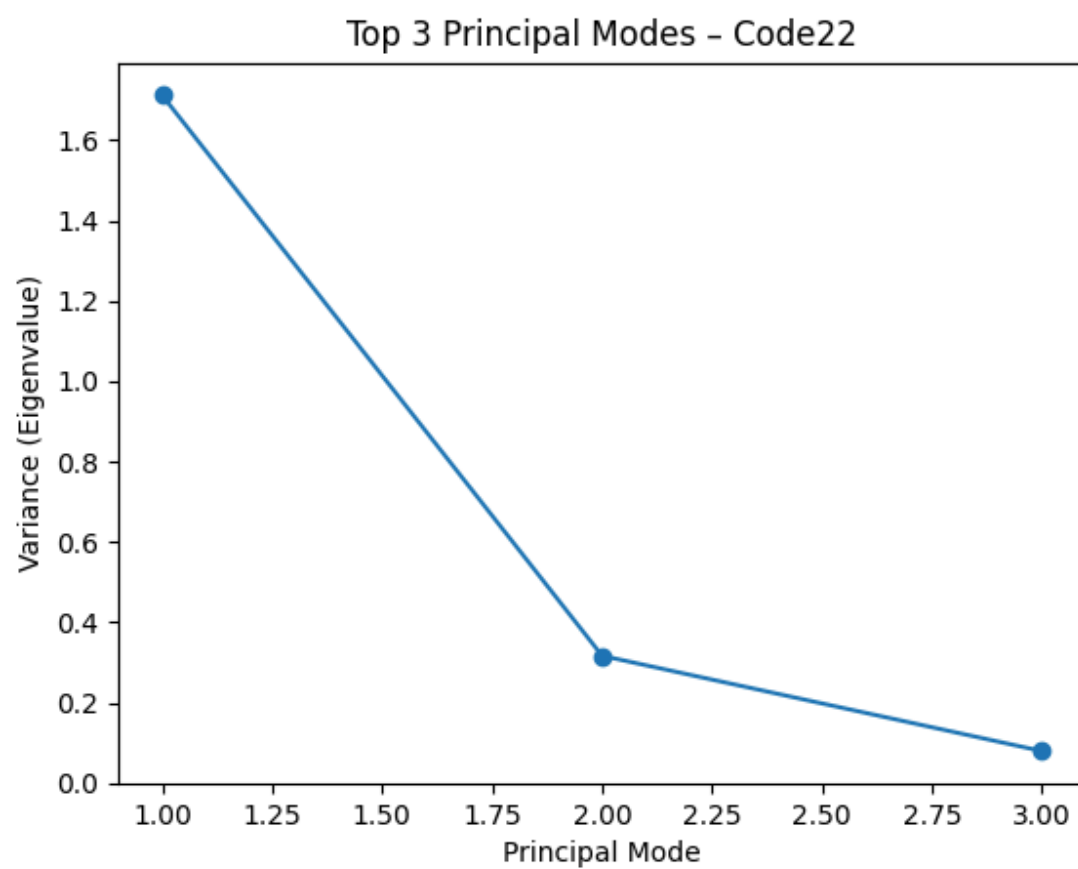


PART D

Code11

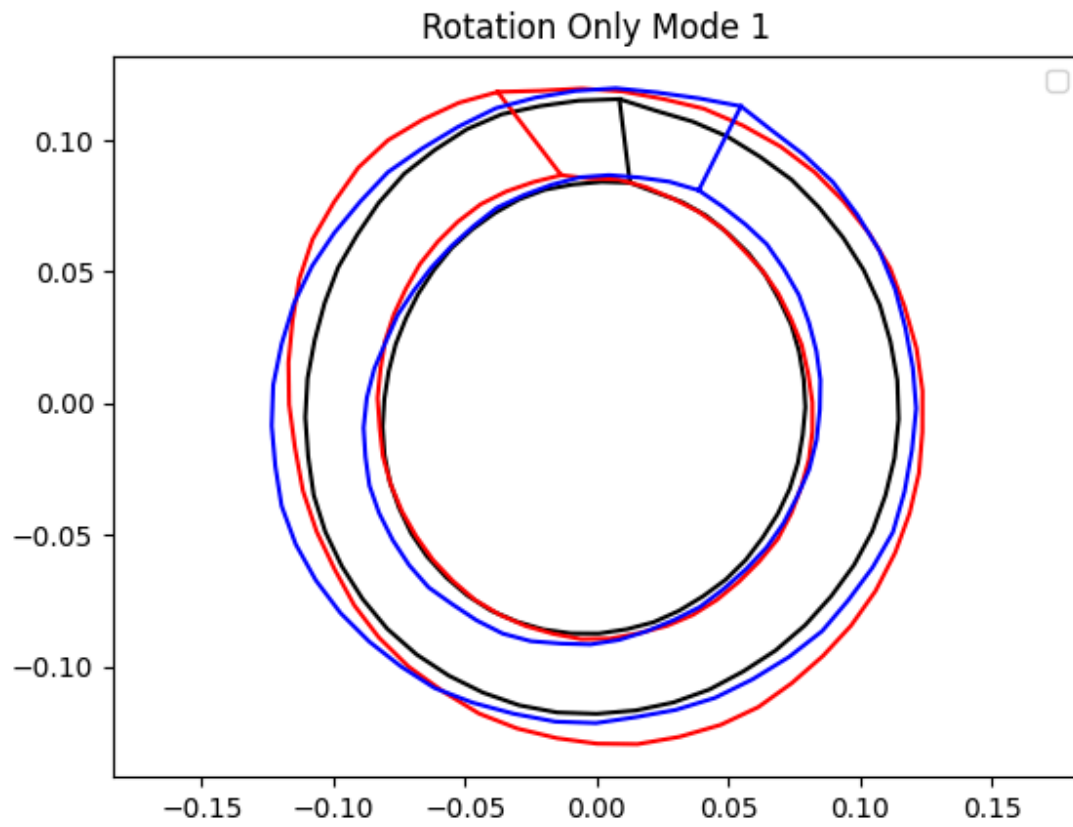


Code22

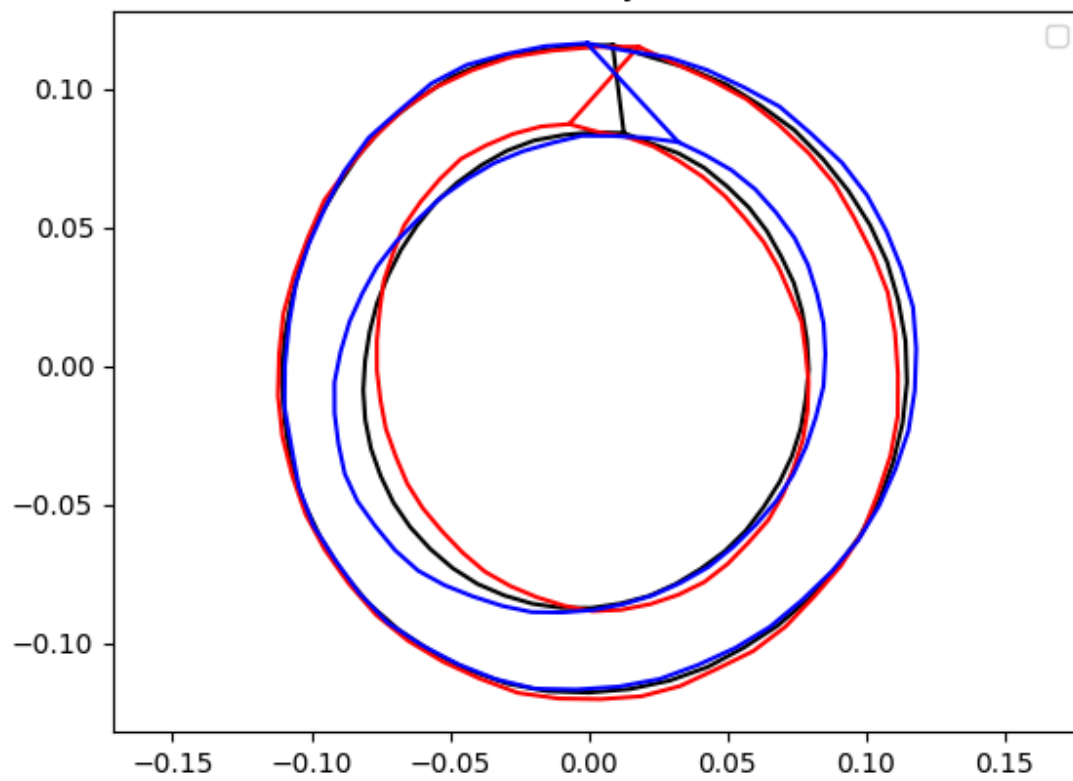


PART E

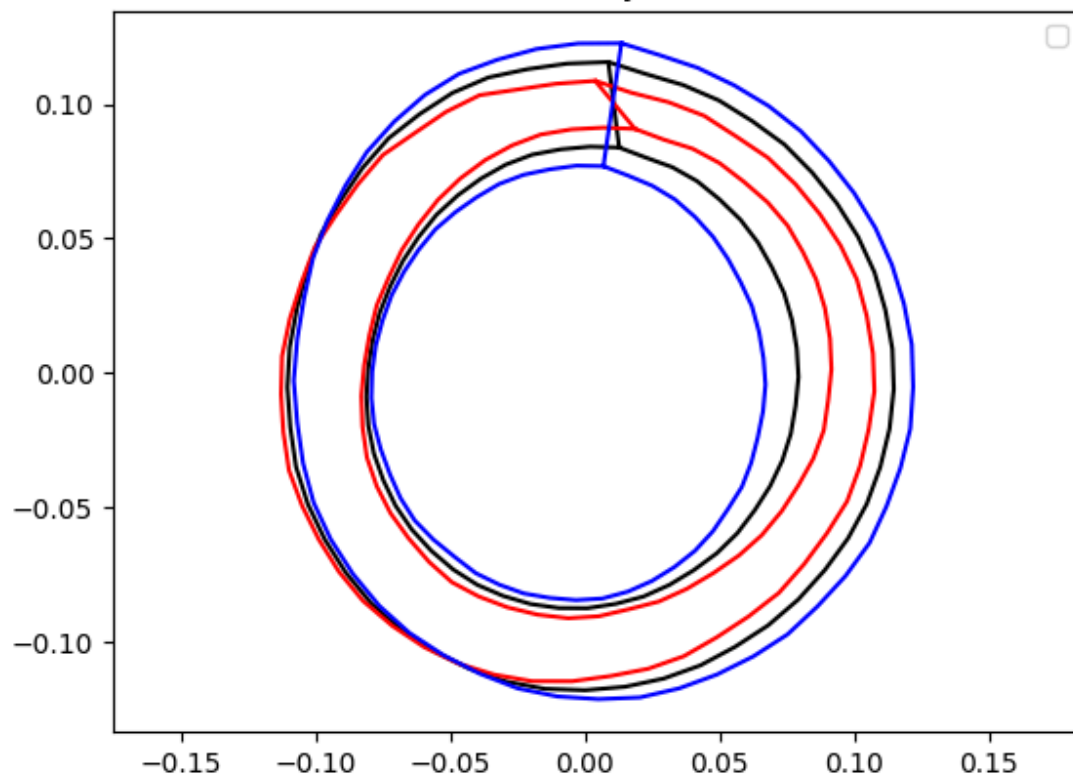
Code11



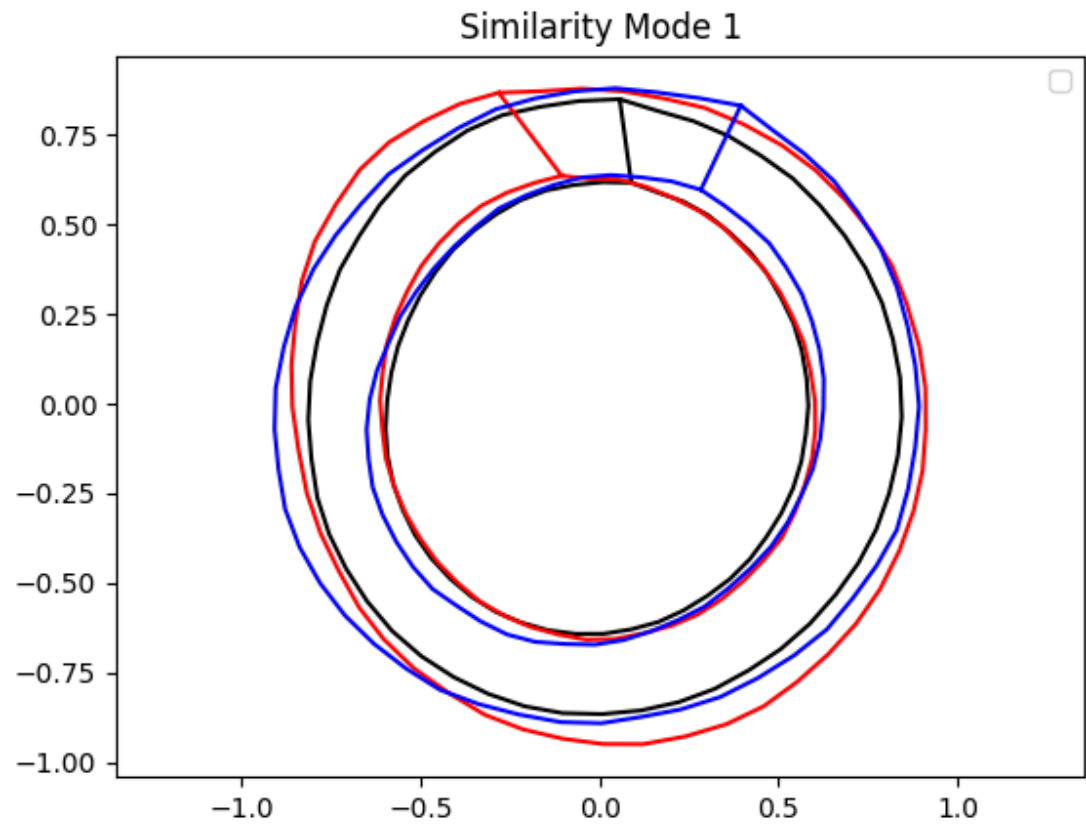
Rotation Only Mode 2



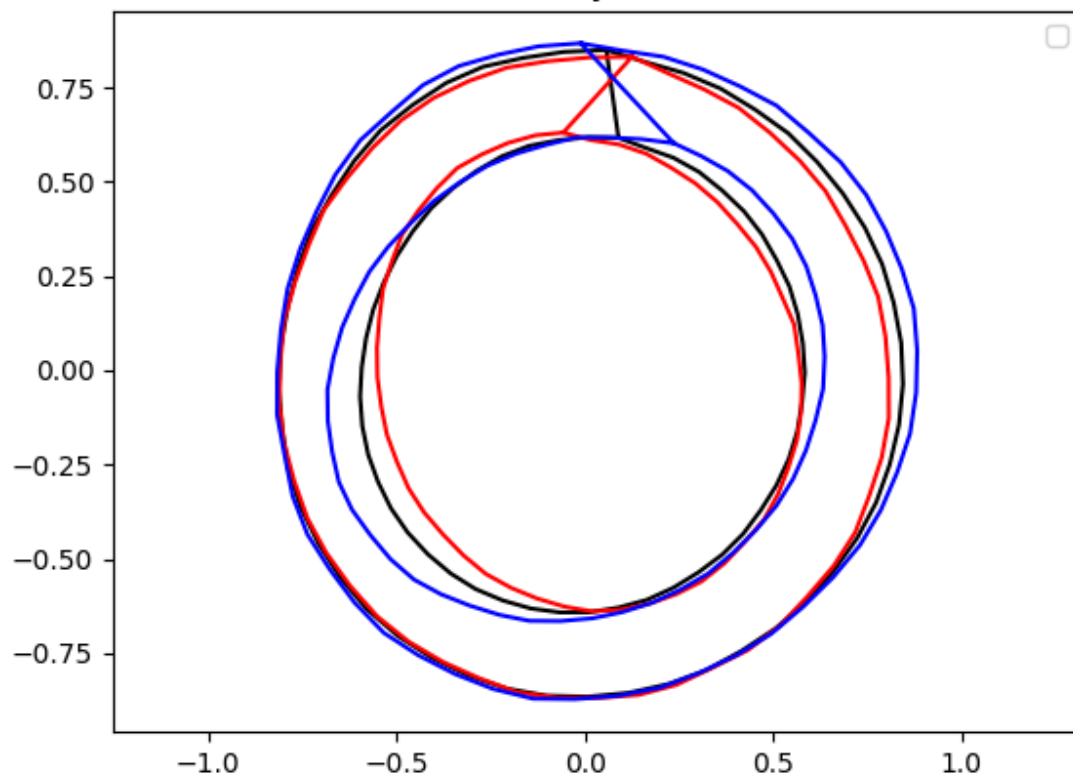
Rotation Only Mode 3



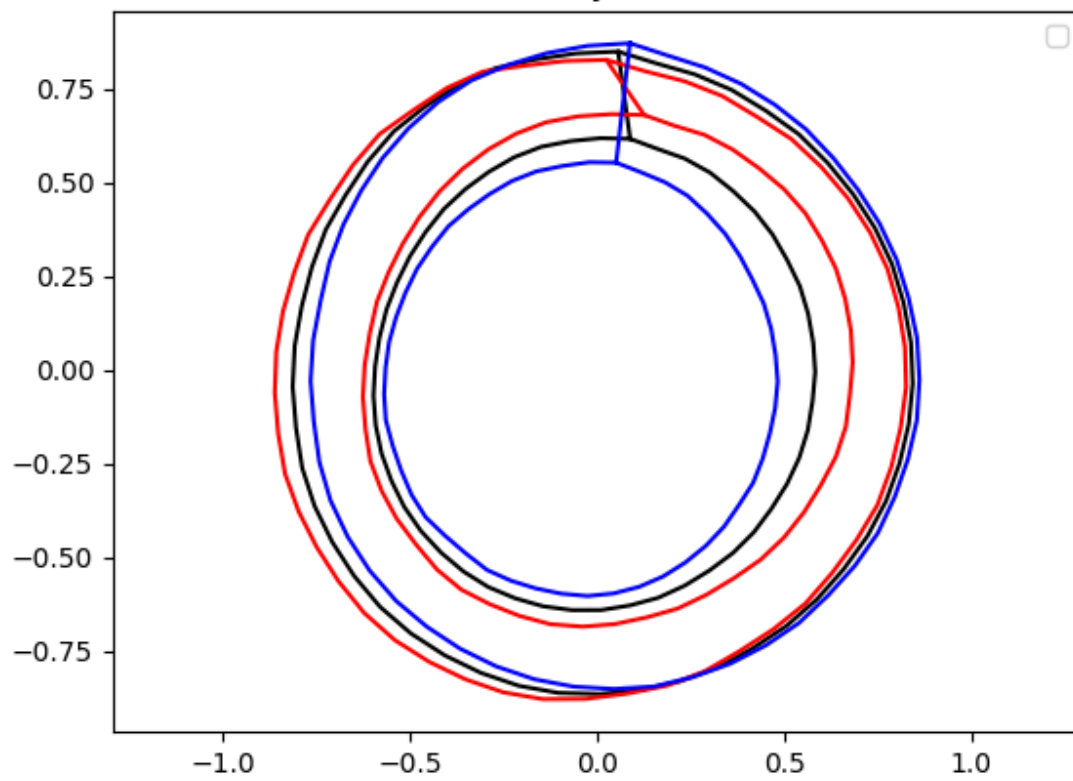
Code22



Similarity Mode 2



Similarity Mode 3



Question 3

Robust Shape Mean

PART A

Design an optimization problem to estimate the mean shape by penalizing the sum of squared Procrustes distances between the mean shape and the individual shapes. For the given dataset, implement a function to compute this mean by solving this optimization problem.

```
def translate(X):
    # X: (32, 2)
    centroid = np.mean(X, axis=0)
    return X - centroid

def scale(X):
    norm = np.linalg.norm(X)
    return X / (norm + 1e-8)

def normalize_shape(X):
    X_centered = X - np.mean(X, axis=0)
    X_centered = X_centered / (np.linalg.norm(X_centered) + 1e-8)
    return X_centered
```

```
import torch
# let's now try to do rotation in an optimal fashion such that it minimizes
the sum of square distance
def optimal_rotation(pointset_1, pointset_2):
    # converting to matrices
    X = torch.Tensor(pointset_1).T
    Y = torch.Tensor(pointset_2).T

    matrix = X @ Y.T
    U, S, Vh = torch.linalg.svd(matrix, full_matrices=True)
    rotation = Vh.T @ U.T
    determinat = torch.linalg.det(rotation)
    if determinat < 0:
        M = torch.eye(2)
        # setting it as -1
        M[1,1] = -1
```

```

        rotation = Vh.T @ M @ U.T

    return rotation,(rotation@X).T

def alignment(X,Z):
    R,X_aligned = optimal_rotation(X,Z)
    dist = ((X_aligned-Z)**2).sum()

    return X_aligned,dist

```

```

def pointset_mean(pointsets):
    # normalizing all pointsets
    normalized_pointsets = []
    for point in pointsets:
        normalized_pointsets.append(normalize_shape(point))

    # choosing initial mean
    idx = np.random.randint(0, len(normalized_pointsets))

    Z = normalized_pointsets[idx]

    while True:
        # step 1: align all points
        aligned_points = []
        for point in normalized_pointsets:
            X,dist = alignment(point,Z)
            aligned_points.append(X.numpy())

        # step 2: compute weighted mean
        aligned_points = np.array(aligned_points)

        mean_shape = aligned_points.mean(axis=0)

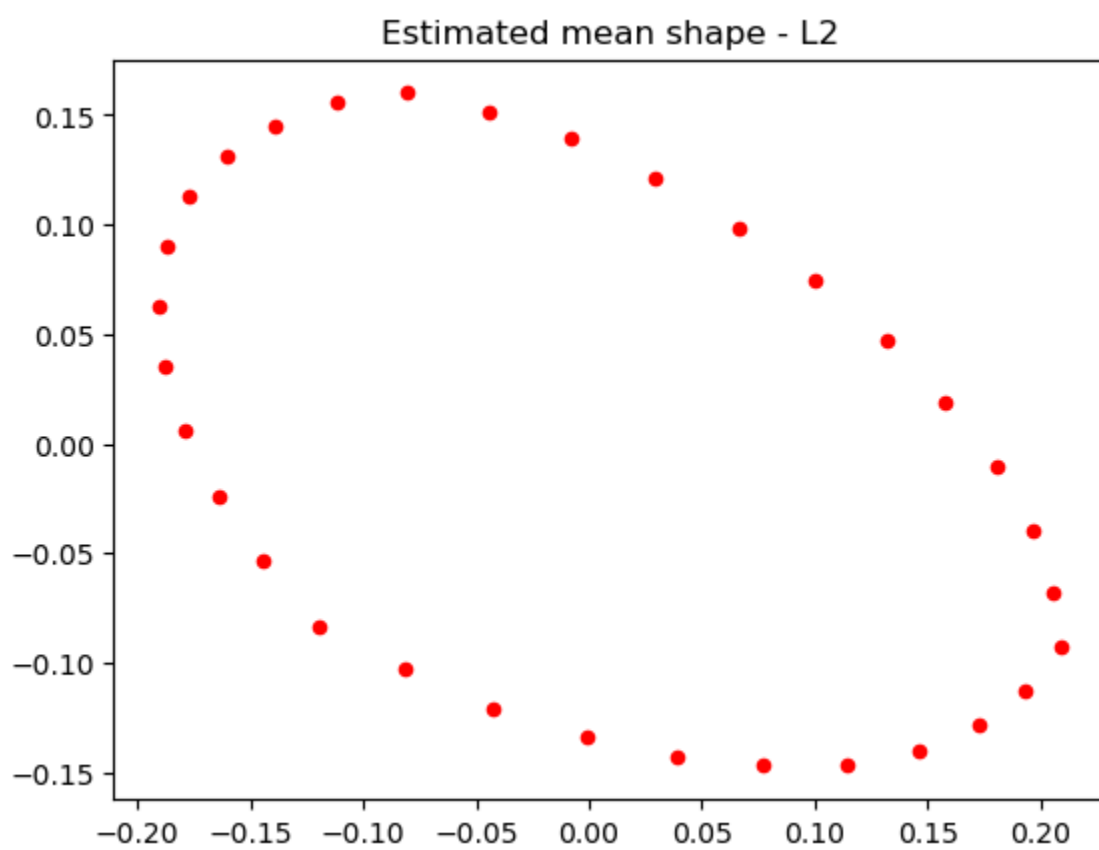
        # project back onto constraint set
        mean_shape = normalize_shape(mean_shape)

        diff = np.linalg.norm(mean_shape - Z)
        print(f"Difference: {diff}")
        if diff<1e-6:
            print("Let's stop")
            break

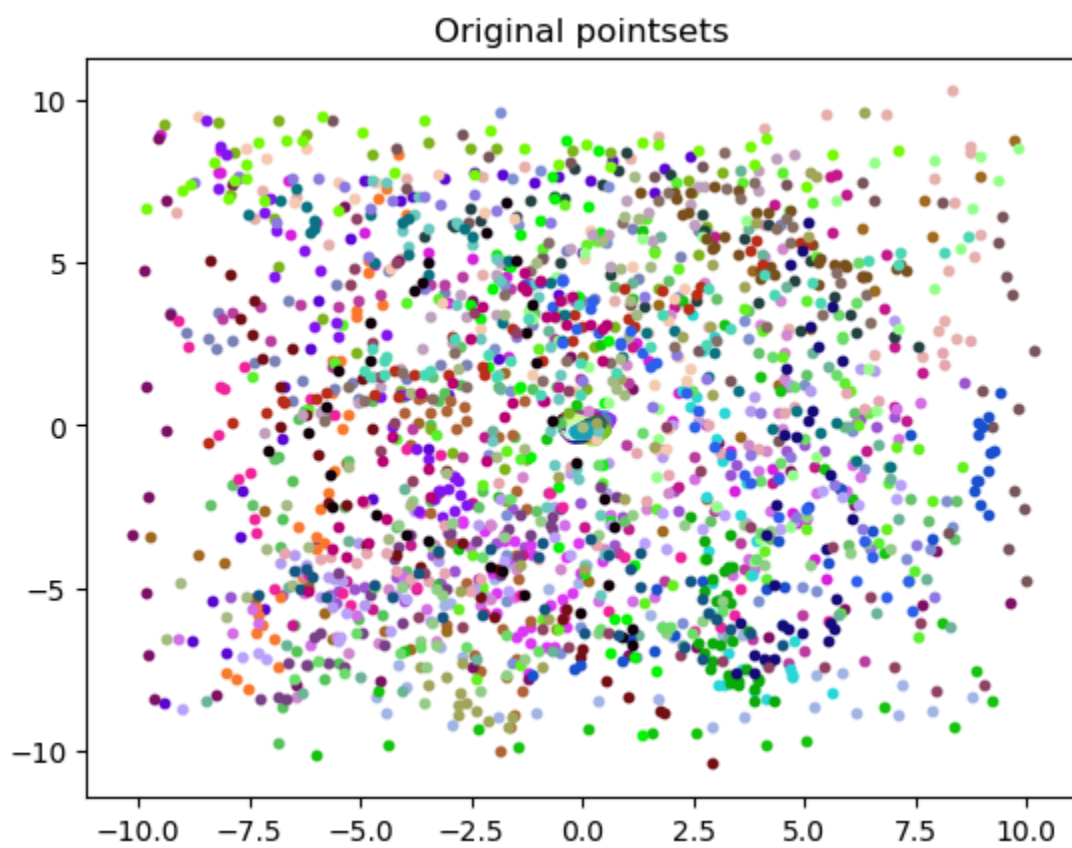
```

```
    Z = mean_shape  
    return Z,aligned_points
```

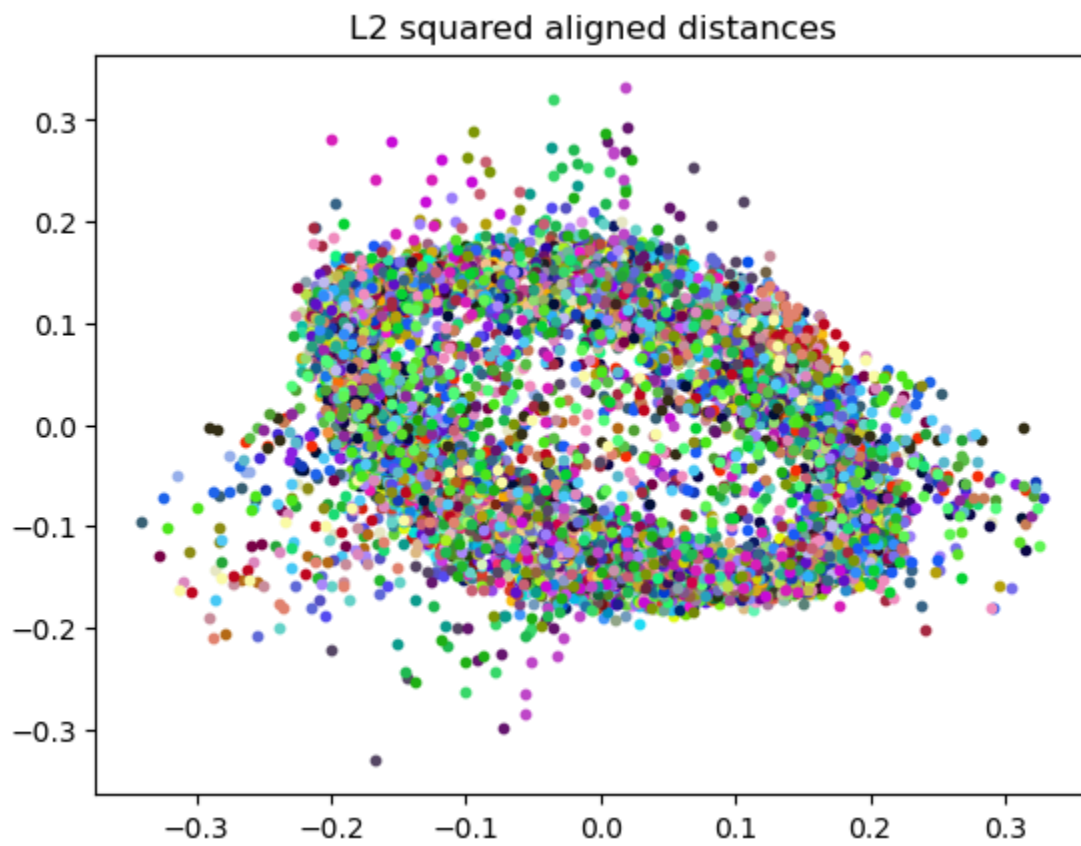
Display the estimated mean



Display the original pointsets.



All original pointsets aligned to the mean.



PART B

```
def rotation_matrix_2d(theta):  
    return np.array([  
        [np.cos(theta), -np.sin(theta)],  
        [np.sin(theta),  np.cos(theta)]  
    ])  
  
def rotation_derivative(theta):  
    return np.array([  
        [-np.sin(theta), -np.cos(theta)],  
        [np.cos(theta),  -np.sin(theta)]  
    ])
```

```
def compute_objective_function(theta,X,Y):  
    R = rotation_matrix_2d(theta)  
  
    total = 0  
    for i in range(len(X)):  
        val = R@X[i] - Y[i]  
        total += np.linalg.norm(val)  
  
    return total  
  
def gradient_wrt_theta(theta,X,Y):  
    R = rotation_matrix_2d(theta)  
    R_grad = rotation_derivative(theta)  
  
    total = 0  
    for i in range(len(X)):  
        val = R@X[i] - Y[i]  
        val_norm = np.linalg.norm(val)  
  
        num = val.T @ (R_grad @ X[i])  
        total += num/(val_norm+1e-8)  
    return total
```

```
import torch  
# let's now try to do rotation in an optimal fashion such that it minimizes  
the sum of square distance
```

```

def optimal_rotation(pointset_1, pointset_2):
    # converting to matrices
    X = np.array(pointset_1)    # (N, 2)
    Y = np.array(pointset_2)    # (N, 2)

    # initializaing theta
    theta = 0
    step_size = 0.005
    prev_obj = compute_objective_function(theta,X,Y)

    for _ in range(200):
        grad = gradient_wrt_theta(theta,X,Y)
        theta = theta - step_size * grad

        new_obj = compute_objective_function(theta,X,Y)

        # print(new_obj)
        if abs(prev_obj - new_obj) < 1e-6:
            break
        prev_obj = new_obj
    rotation = rotation_matrix_2d(theta)

    return rotation,(rotation@X.T).T

def alignment_l2(X,Z):
    R,X_aligned = optimal_rotation(X,Z)
    # print(X_aligned.shape)
    dist = ((X_aligned-Z)**2).sum()

    return X_aligned,np.sqrt(dist)

```

```

def weighted_mean(aligned_shapes, weights):
    mean_shape = np.zeros_like(aligned_shapes[0])

    for i in range(len(aligned_shapes)):
        mean_shape += weights[i] * aligned_shapes[i]

    mean_shape /= weights.sum()
    return mean_shape

def robust_mean(pointsets):

```

```

# normalizing all pointsets
normalized_pointsets = []
for point in pointsets:
    normalized_pointsets.append(normalize_shape(point))

# choosing initial mean
idx = np.random.randint(0, len(normalized_pointsets))

Z = normalized_pointsets[idx]

while True:
    # step 1: align all points
    aligned_points = []
    weights = []
    for point in normalized_pointsets:
        X,dist = alignment_l2(point,Z)
        aligned_points.append(X)
        weights.append(1.0/(dist+1e-8))

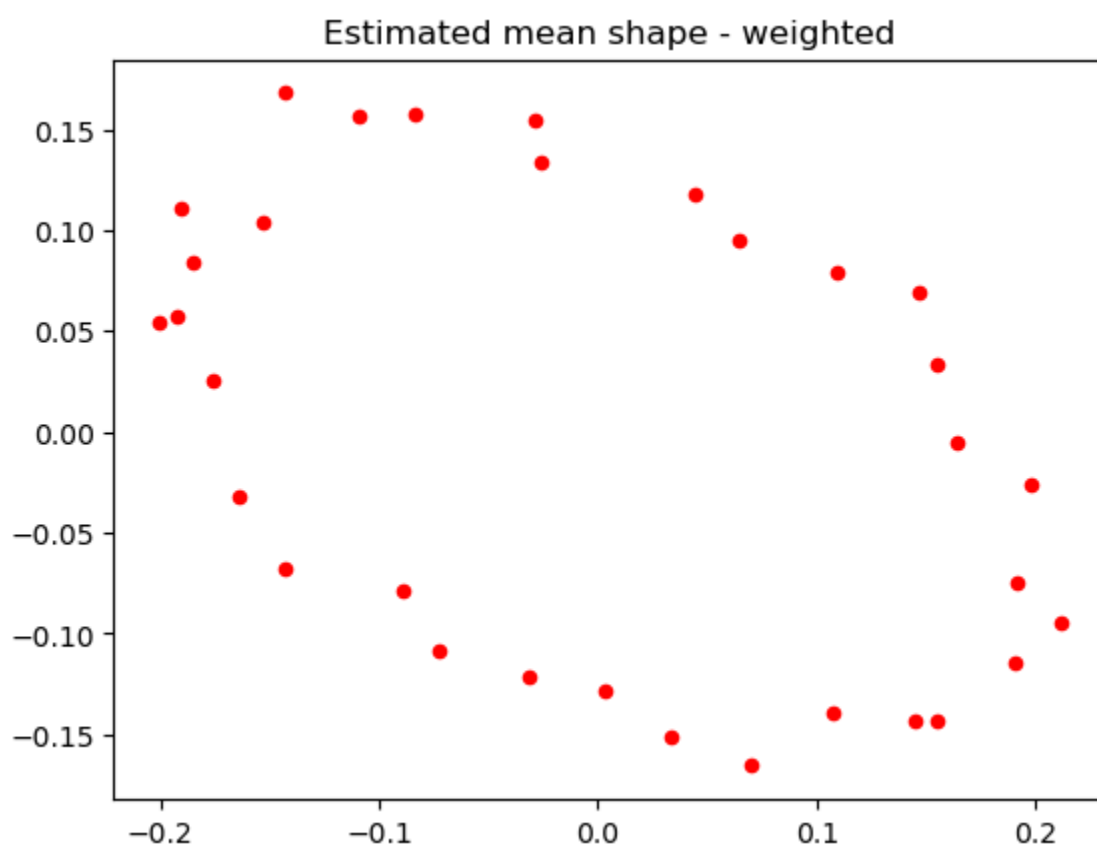
    # step 2: compute weighted mean
    mean_shape =
weighted_mean(np.array(aligned_points),np.array(weights))

    # project back onto constraint set
    mean_shape = normalize_shape(mean_shape)

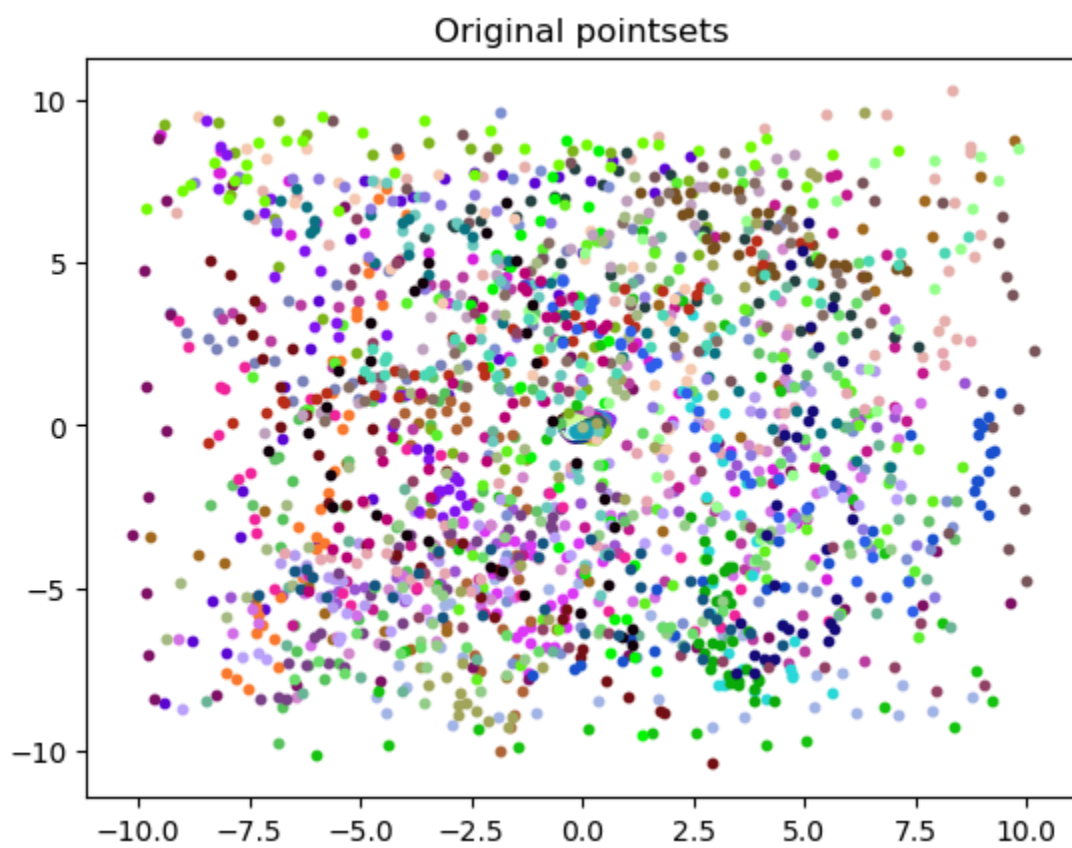
    diff = np.linalg.norm(mean_shape - Z)
    print(f"Difference: {diff}")
    if diff<1e-3:
        print("Let's stop")
        break
    Z = mean_shape
return Z,aligned_points

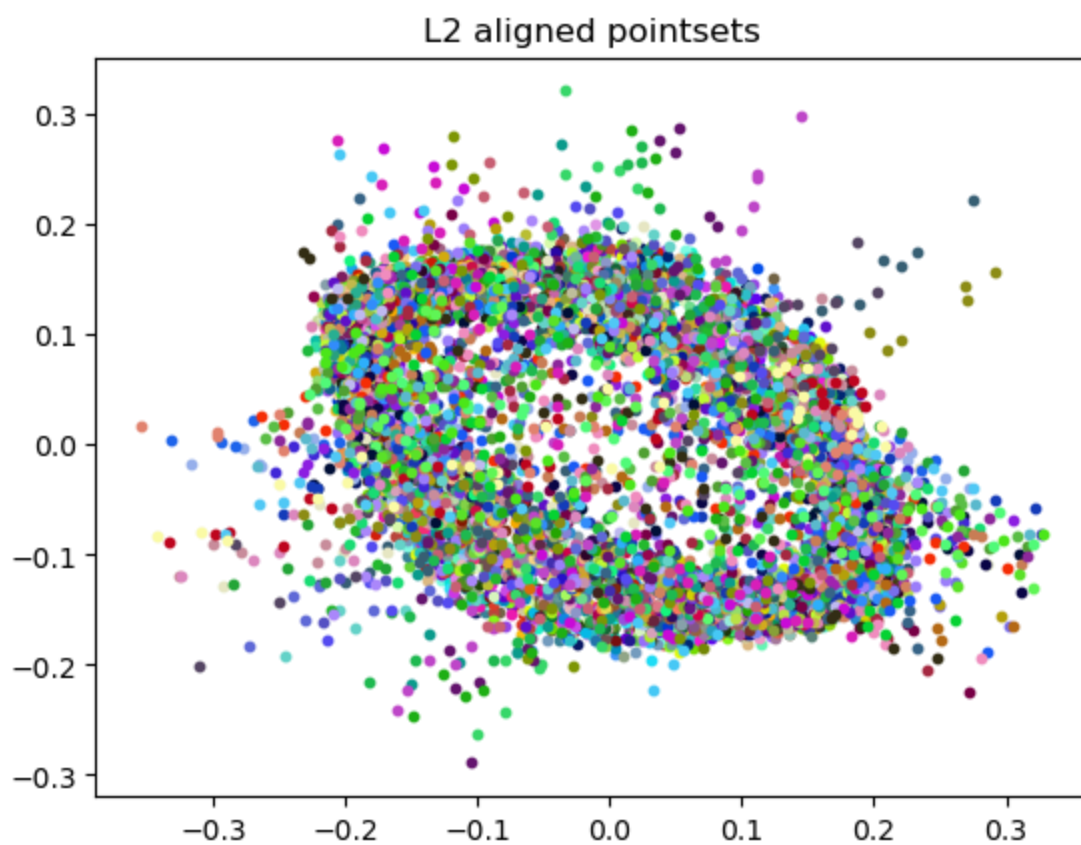
```

Display the estimated mean



Display the original pointsets.





all original pointsets aligned to the mean

Question 4

Kernel PCA to Model Variation in Object Segmentations.

Pre-processing functions:

```
def min_max_normalize(image):
    arr = np.array(image).astype(np.float32)

    min_val = arr.min()
    max_val = arr.max()

    normalized = (arr - min_val) / (max_val - min_val + 1e-8)

    return normalized

def preprocess_image(image_path):

    if image_path.endswith(".png"):
        img = Image.open(image_path)
        img_resized = img.resize((64, 64))
        normalized = min_max_normalize(img_resized)
        return normalized
```

PART A

Implement a function to perform PCA on the (vectorized) segmentation images.

Function :

```
def get_mean(images):
    return images.mean(axis=0)

mean =get_mean(vec_segs)

# performing PCA
def perform_pca(vec_segs,mean):
    # centered data
    centered_X = vec_segs - mean[None,:]

    # computing covariance matrix - it is across features and not data
    points
    cov =(centered_X.T @ centered_X)/centered_X.shape[0]

    # computing eigen values and eigen vectors of covariance matrix
    eg_pca , ev_pca = np.linalg.eigh(cov)

    # sorting eigen values
    idx = np.argsort(eg_pca)[::-1]
    eg_pca = eg_pca[idx]
    ev_pca = ev_pca[:,idx]

    return eg_pca,ev_pca

eg_pca,ev_pca = perform_pca(vec_segs,mean)

# modes of variation - models the thickness of the ring
i = 0
mov = [mean - 2*np.sqrt(eg_pca[i])*ev_pca[:,i], mean +
2*np.sqrt(eg_pca[i])*ev_pca[:,i]]

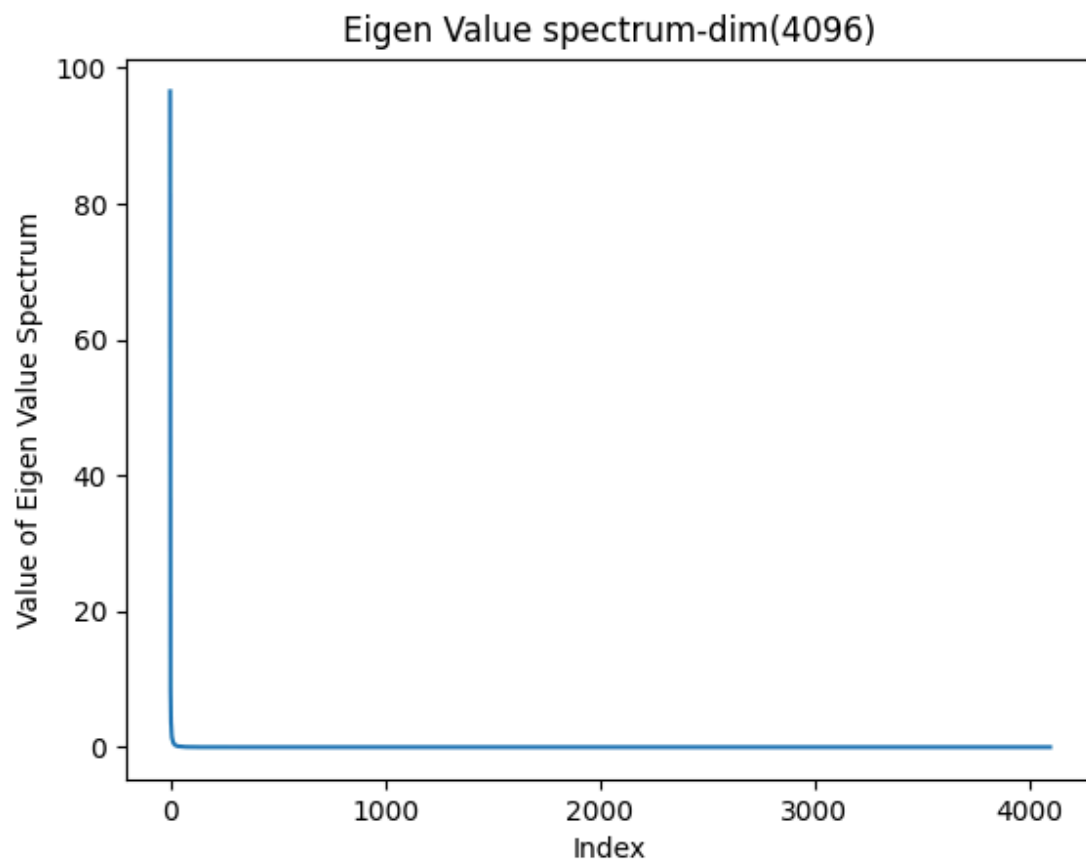
# compute shared scale
vmin = min(mov[0].min(), mean.min(), mov[1].min())
vmax = max(mov[0].max(), mean.max(), mov[1].max())

# modes of variation - models the intensity difference of the center ring,
```

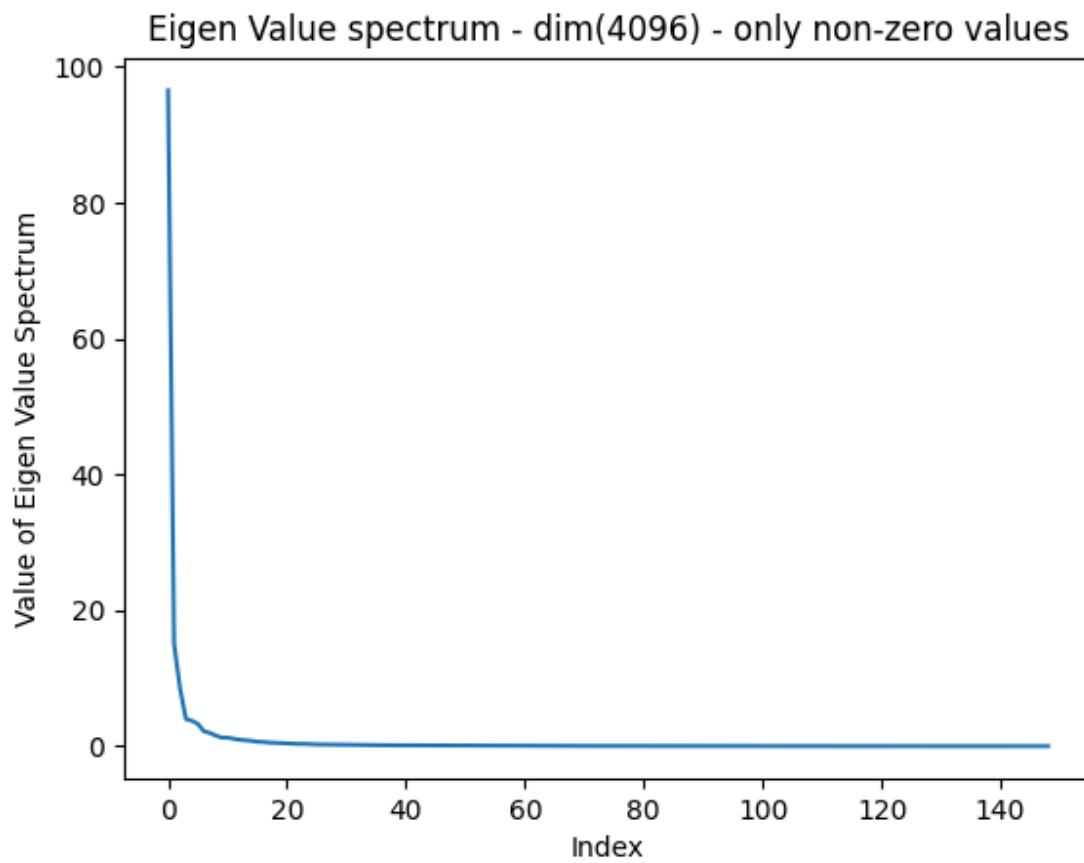
```
brighter and dimmer across means
i = 1
mov = [mean - 2*np.sqrt(eg_pca[i])*ev_pca[:,i], mean +
2*np.sqrt(eg_pca[i])*ev_pca[:,i]]

# compute shared scale
vmin = min(mov[0].min(), mean.min(), mov[1].min())
vmax = max(mov[0].max(), mean.max(), mov[1].max())
```

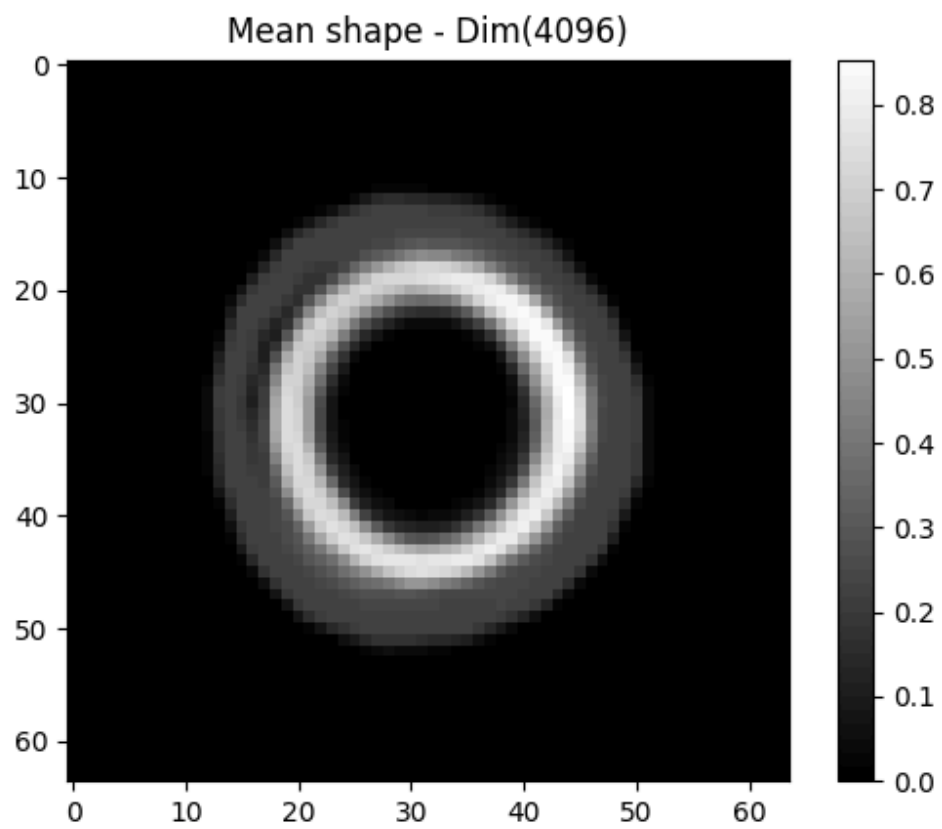
Show (i) the eigen spectrum



Extra visualization : The matrix only has rank ≤ 150 and hence the number of non-zero eigenvalues is also ≤ 150

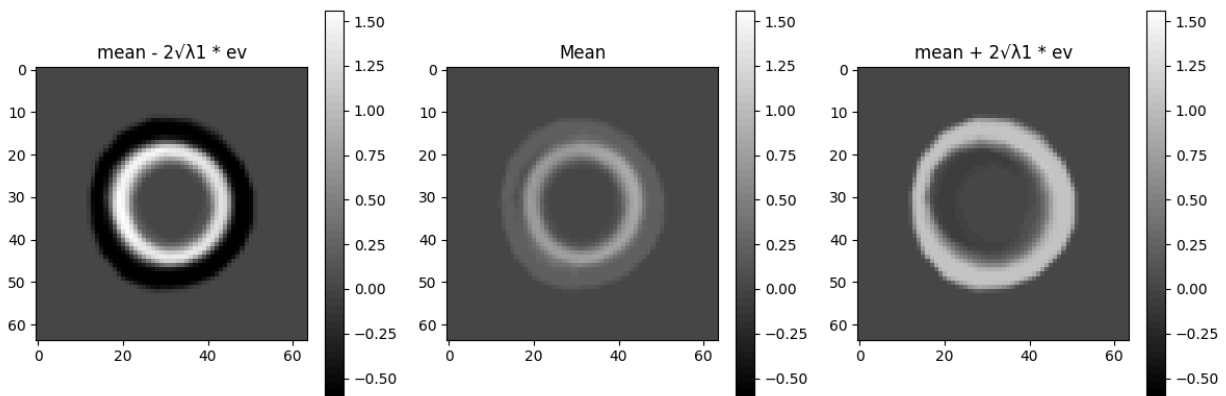


(ii) the mean image

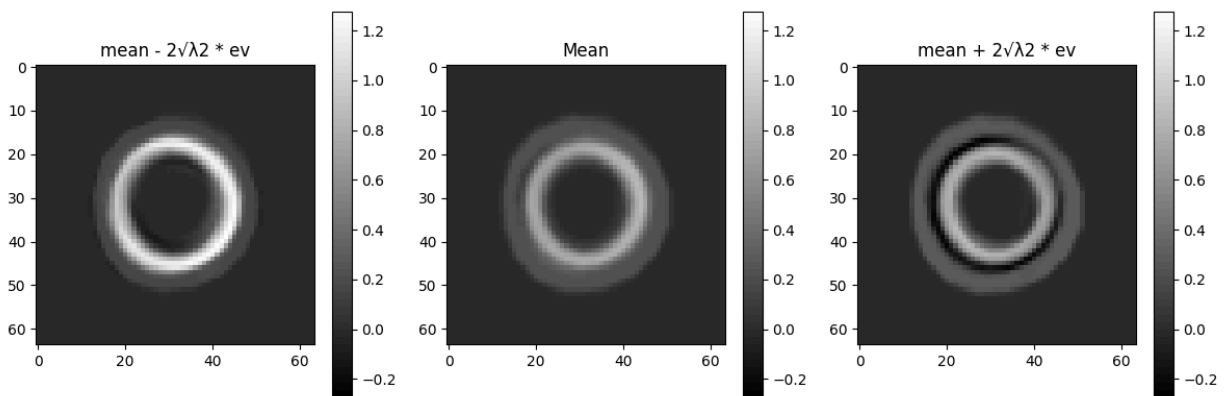


(iii) the first 2 modes of variation around the mean (as images).

First mode of variation: models the thickness of the ring and diameter of ring



Second mode of variation: models the intensity difference of the center ring, brighter and dimmer across means



PART B

Using a Gaussian kernel between the (vectorized) segmentation images, implement a function to perform kernel PCA.

Function:

```
def compute_inner_product(x1,x2,sigma=1):  
    return np.exp((-0.5*np.linalg.norm(x1-x2)**2)/(sigma**2))
```

```
def compute_eg_spectrum(sigma=1):  
    # kernel matrix  
    K = np.zeros(shape=(150,150))  
    N = X.shape[0]  
  
    for i in range(N):  
        for j in range(N):  
            K[i][j] = compute_inner_product(X[i],X[j],sigma=sigma)  
  
    # let's mean center the data here  
    N = K.shape[0]  
    one_N = np.ones((N, N)) / N  
  
    # subtract that particular row and column sum of values and add overall  
    sum of all values  
    K_centered = K - one_N @ K - K @ one_N + one_N @ K @ one_N  
  
    # performing PCA on kernel matrix  
    eg,ev = np.linalg.eigh(K_centered)  
  
    # sorting eigen values  
    idx = np.argsort(eg)[::-1]  
    eg = eg[idx]  
    ev = ev[:,idx]  
  
    # getting eigen values of kernel matrix  
    # this is because of eigen vector being linear combination of spanned  
    data where linear combination is my eigen vector coefficients  
    ev_unit_norm = ev / np.sqrt(np.maximum(eg, 1e-10))[None, :]  
  
    # Now divide eg by N for the spectrum  
    eg = eg / N
```

```

plt.plot(eg)
plt.xlabel("Index")
plt.ylabel("Value of EV")
plt.title("Eigen Value for Kernel PCA")
plt.show()

```

```

return eg, ev_unit_norm, K

```

```

def compute_pre_image(sigma=1):
    print(sigma)
    K = np.zeros(shape=(150,150))
    N = X.shape[0]

    for i in range(N):
        for j in range(N):
            K[i][j] = compute_inner_product(X[i],X[j],sigma=sigma)

    # pre-estimate of image
    # point closet to mean in kernel space from our dataset
    # min || mean - phi(x)|| = max inner_product(mean,phi(x))
    closet_to_mean = (K.sum(axis=1)).argmax()

    # initializing our estimate of mean to be the closet point in our
dataset
    initial_mean = X[closet_to_mean]
    initial_objective_value =
np.array([compute_inner_product(initial_mean,x,sigma=sigma) for x in
X]).sum()
    print(f" {initial_objective_value} - Initial Objective Value")

    # computing via iterative method
    max_iters = 200
    objective_value = None
    for i in range(max_iters):
        # new mean
        k_x = np.array([compute_inner_product(initial_mean,x,sigma=sigma)
for x in X])
        new_mean = (k_x[:, None] * X).sum(axis=0) / (k_x.sum() + 1e-10)

        objective_value =
np.array([compute_inner_product(new_mean,x,sigma=sigma) for x in X]).sum()

```

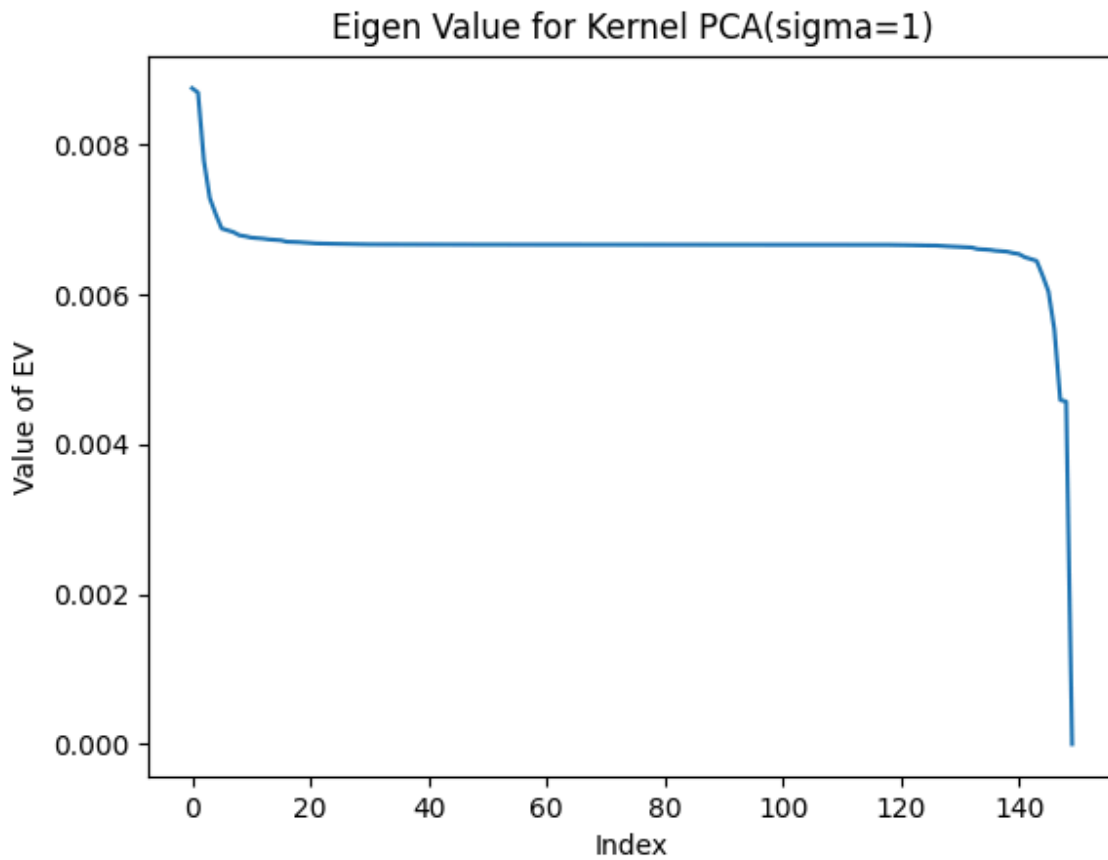
```
    if np.linalg.norm(new_mean-initial_mean)<1e-6:
        print(f"{i} breaking now")
        break
    #upating the value
    initial_mean = new_mean

print(f"{objective_value} : Final objective value")
print(f"Change in objective:
{objective_value-initial_objective_value}")
plt.imshow(new_mean.reshape(64,64),cmap="gray")
plt.title(f"Pre-image sigma({sigma})")
plt.show()

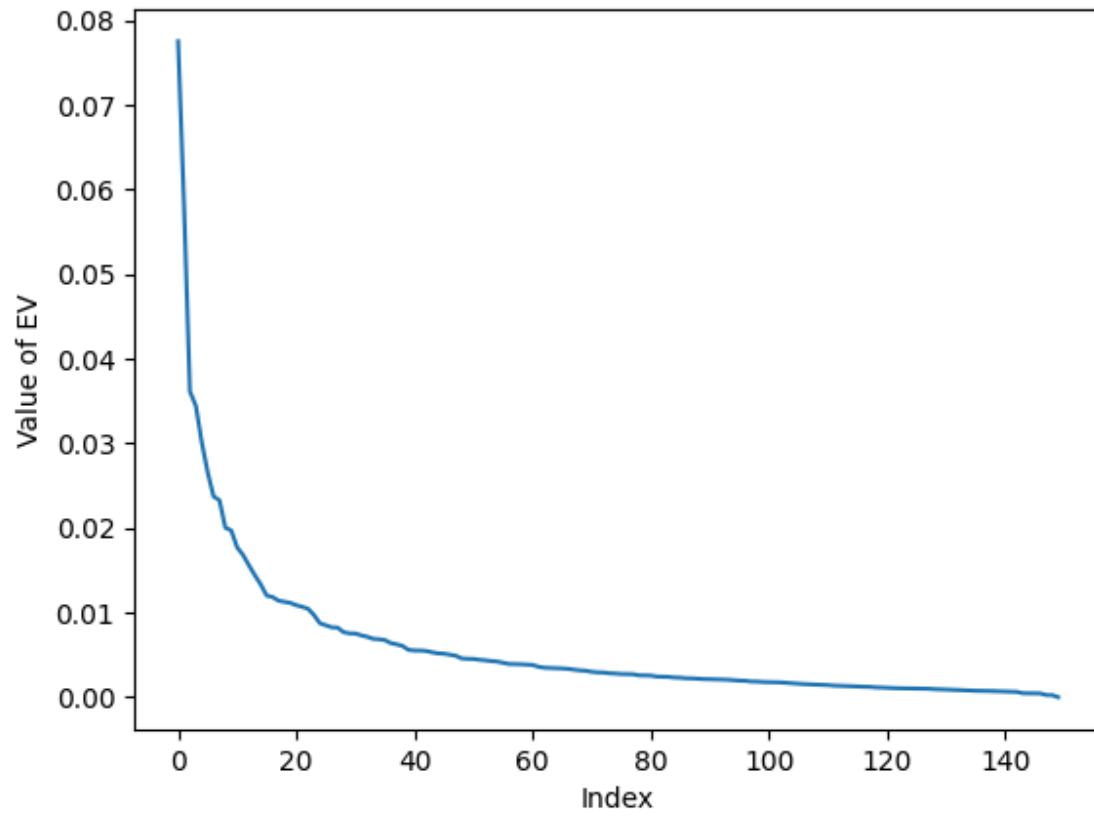
return objective_value-initial_objective_value
```

(i) the eigen spectrum

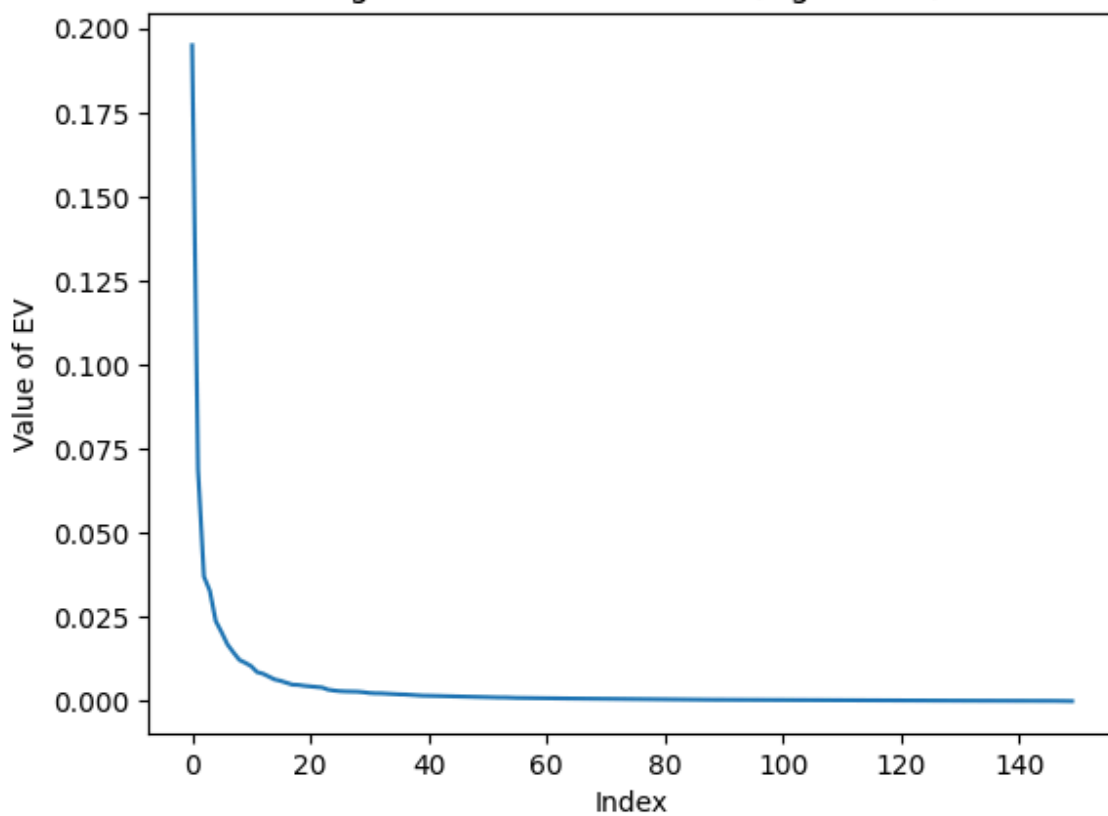
Here we are adding the eigen value for different sigma's, there is no criteria as such to find optimal value of sigma based on eigen values, we leave that to part C of the question



Eigen Value for Kernel PCA($\sigma=5$)



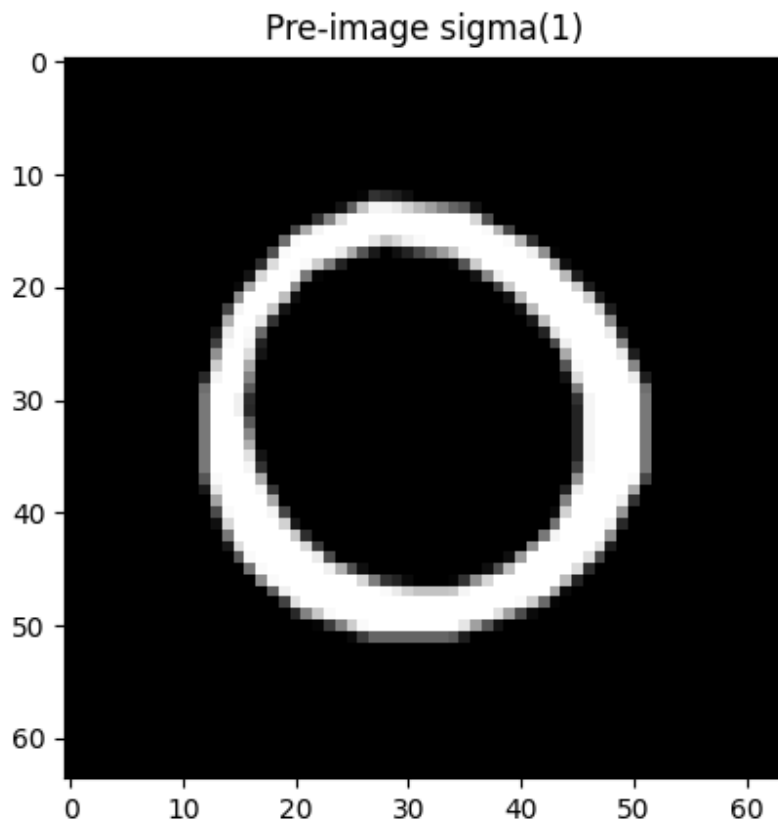
Eigen Value for Kernel PCA(sigma=10)

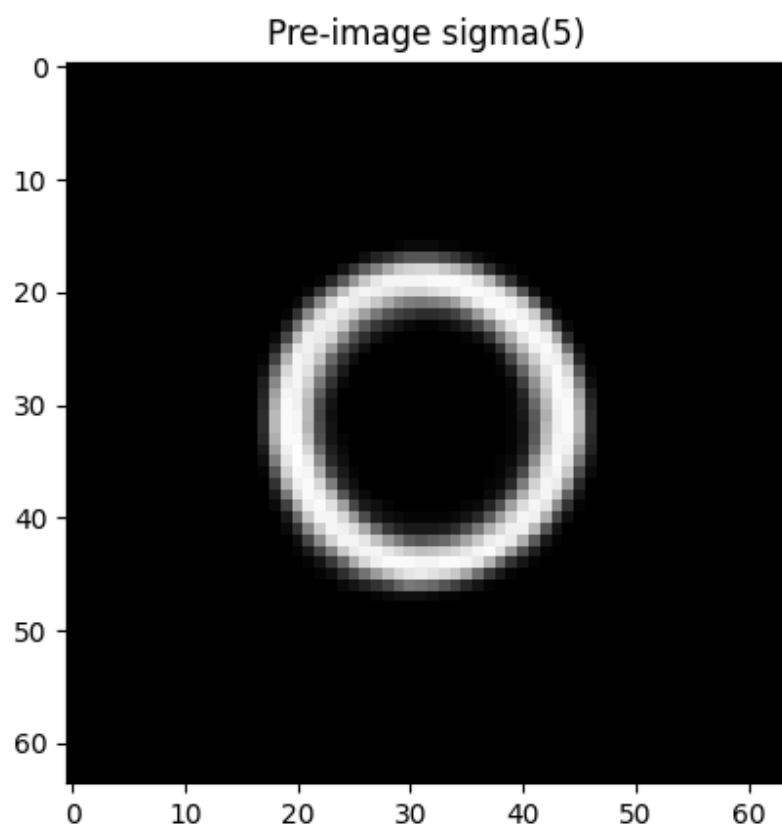


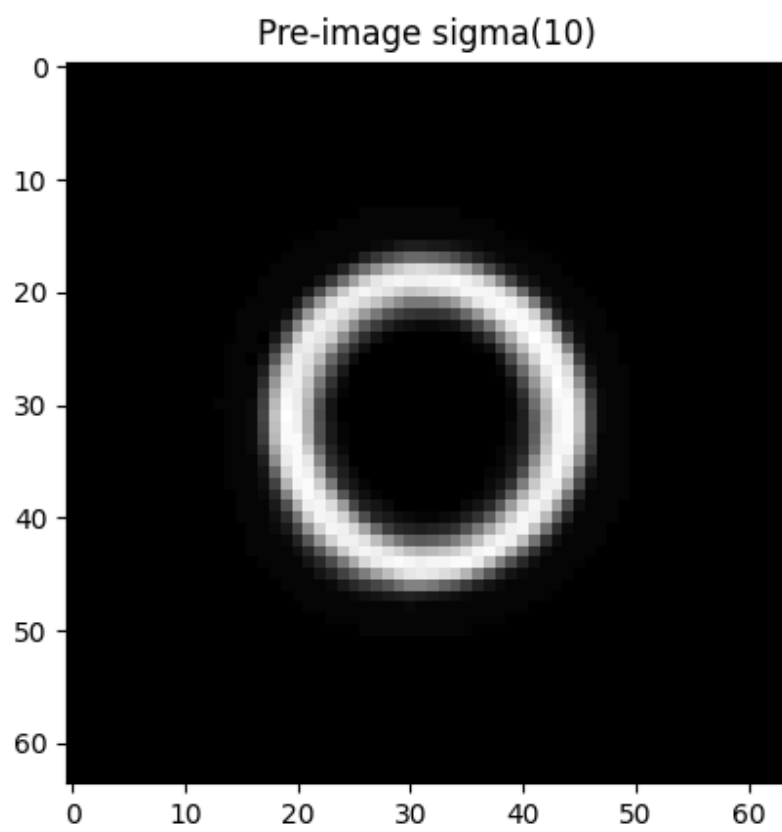
(ii) an estimate of the segmentation image that the kernel would (implicitly) map to a point closest to the mean in RKHS; such an estimate is called the pre-“image” associated with the kernel.

As sigma goes higher, we are essentially considering more points in our estimate i.e. it will become more smooth, as visible below.

If $\sigma \rightarrow 0$, then only a small number of values with minimal distances are considered whereas otherwise high values i.e. points far away are also considered, making the image smoother.







PART C

For each segmentation in “anatomicalSegmentationsDistorted.zip”, implement functions to aim to reduce the distortions by projecting the segmentation images onto the first 3 modes of variation in:

(i) PCA, and display the projected segmentation images

```
def projected_image_pca(input_image, eigen_values, eigen_vectors, mean, k=3):
    # assuming that eigen vectors and eigen values are ordered
    projected_vector = 0
    for i in range(k):
        projected_vector += (eigen_vectors[:,i]@(input_image-mean)) *
eigen_vectors[:,i]
    reconstruction = projected_vector + mean
    return reconstruction
```

(ii) kernel PCA; here, the projections will be in RKHS; for each of the aforementioned projections in RKHS, implement a function to compute its pre-“image” as a segmentation image and display it

```
def projection_using_kpca(vector, ev, eg, K, sigma=1, k=3):
    # computing value for kernel
    k_x = np.array([compute_inner_product(xi, vector, sigma) for xi in X])

    # centering the kernel
    k_x_centered = k_x - K.mean(axis=1) - k_x.mean() + K.mean()

    # now computing inner product
    inner_product = ev[:, :k].T @ k_x_centered
    print(inner_product[:3])

    # now computing the scaling with eigen vectors
    beta = ev[:, :k] @ inner_product # inner product is (3,)

    # Pre-image
    closet_to_projection = (K@beta).argmax()
    x = X[closet_to_projection].copy()

    # computing initial objective
```

```

    initial_objective_value = np.array([beta[i] * compute_inner_product(x,
xi, sigma) for i, xi in enumerate(X)]).sum()
    print(f"{initial_objective_value} - Initial Objective Value")

    max_iters = 200
    objective_value = None

    for i in range(max_iters):
        k_x_new = np.array([compute_inner_product(x, xi, sigma) for xi in
X])

        weights = beta * k_x_new

        x_new = (weights[:, None] * X).sum(axis=0) / (weights.sum() +
1e-10)

        # computing the objective
        objective_value = np.array([
            beta[j] * compute_inner_product(x_new, xj, sigma) for j, xj in
enumerate(X)
        ]).sum()

        if np.linalg.norm(x_new - x) < 1e-6:
            print(f"{i} breaking now")
            break

        x = x_new

    print(f"{objective_value} : Final objective value")
    print(f"Change in objective: {objective_value -
initial_objective_value}")

    return x

```

Code for finding optimal value of sigma:

```

errors = []
for sigma in [0.5,1,2,5,9,10,15,20]:
    print("-"*100)
    print("Sigma is", sigma)
    eg,ev_unit_norm,K = compute_eg_spectrum(sigma=sigma)

```

```
# projection onto top 3 eigen vectors
index = 25
error = 0
for vector in X_distored:
    vector = vector.reshape(-1)
    recons =
projection_using_kpca(vector,ev_unit_norm,eg,K,sigma=sigma,k=3)
    error += np.mean((vector - recons)**2)
error /= len(X_distored)
print(f"{error} for sigma : {sigma}")
print("-"*100)
errors.append(error)
```

Optimal Value of sigma: 10

Visualization for all 150 images is added in the pdf.